Why the Exponential Dimension Matters

- Classical simulator needs 2^n complex numbers; quantum computer uses n qubits
- Patterns in this space that are classically hard to simulate can only be learned by a quantum model
- This is **representational advantage** — not about speed, about what functions can be represented

Important Caveat

Exponential dimension $\neq$ better learning. A feature map that makes classes harder to separate is worse than a simple classical one. Advantage is task-dependent and an active research frontier.

Geometry of the Quantum Feature Space

- Each encoded point is a unit vector in $\mathbb{C}^{2^n}$
- Similarity measured by the **quantum kernel**: $K(x, z) = |\langle\psi(x)|\psi(z)\rangle|^2$
- Squared magnitude - inner product is complex; squaring gives real score in $[0, 1]$
- Kernel trick: never work in the full 2^n space — only compute pairwise inner products
- Estimated on hardware via Hadamard test or swap test — efficient on quantum, exponentially expensive classically

Choice of encoding = choice of geometry = what can be separated.

What Kernel Does Each Encoding Induce?

Encoding	Feature map	Kernel
Amplitude	identity (after normalise)	$\lvert x^\dagger z\rvert^2$ - squared linear
Basis	one-hot in $\mathbb{R}^{2^n}$	$\delta_{x,z}$ - no generalisation
Angle	trigonometric products	$\prod_k \lvert\cos(x_k - z_k)\rvert^2$
Hamiltonian	Fourier + interactions	interaction-dependent

- **Amplitude**: preserves original geometry
- **Basis**: identical points only - cannot generalise
- **Angle**: smooth, periodic - good for continuous data
- **Hamiltonian**: captures joint feature structure

Choice of encoding = choice of kernel = what can be learned.

Designing a Good Feature Map

A good quantum feature map must:

- **Capture discriminative structure** — different classes far apart; same class nearby
- **Include relevant interactions** — joint feature structure requires multi-qubit terms; single-qubit encodings cannot represent correlations
- **Be implementable on hardware** — circuit depth must match device coherence time and noise

No principled method exists for finding the optimal feature map — same open problem as kernel selection in classical SVM.

The Complete QML Pipeline

$$x \xrightarrow{\text{load}} |x\rangle \xrightarrow{U(x)} |\psi(x)\rangle \xrightarrow{\text{measure}} \text{output}$$

- **Load**: $x \to |x\rangle$ via X gates — physical input interface
- **Encode**: $U(x)\,|0\rangle$ - creates the geometric structure; this is the feature map
- **Measure**: expectation value of an observable — prediction or kernel value

Output quality is determined entirely by the encoding step.

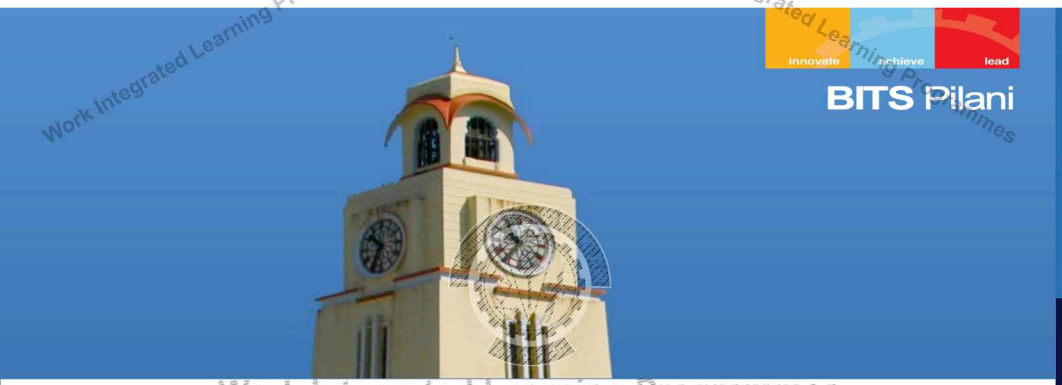
Quantum ML = encoding (feature map) + measurement.

Summary and Transition

- Every encoding is a feature map $x \to |\psi(x)\rangle = U(x)\,|0\rangle$
- Quantum kernel $K(x, x') = |\langle\psi(x)|\psi(x')\rangle|^2$ measures similarity in feature space
- Each encoding → specific kernel → specific geometry → specific learnable patterns
- Encoding is the most fundamental design decision in QML

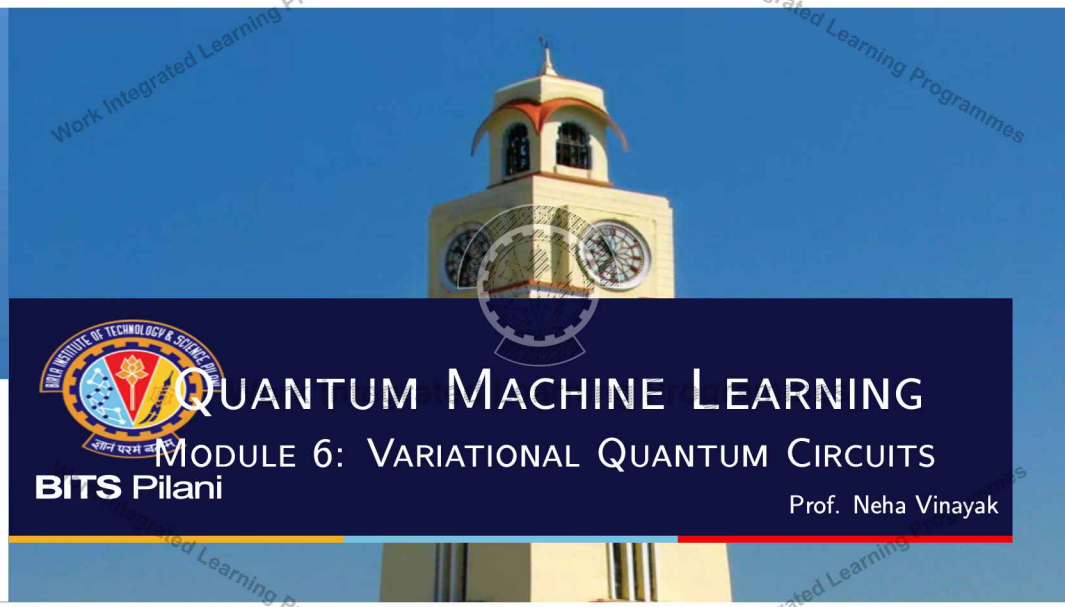
Next: how do we build learning algorithms on top of these feature maps?

Next Module Variational Quantum Circuits - encoding layer $U(x)\,|0\rangle$ combined with trainable layer $W(\theta)$.



BITS Pilani

Thank you :)



Quantum Machine Learning

Module 6: Variational Quantum Circuits

BITS Pilani

Prof. Neha Vinayak

Table of Contents

Module 6: Session Plan

Session 9 (Today)

1. Why variational circuits?
2. Circuits as ML models
3. Deterministic quantum models
4. Probabilistic quantum models
5. Variational quantum classifiers
6. Variational quantum generators

Session 10

1. Expressibility of quantum models
2. Gradients of quantum computations
3. Parameter-shift rules
4. Barren plateaus and trainability
5. Quantum circuits and neural networks

Can a quantum circuit play the role of a machine learning model f_θ?

A Shift in Research Objective

Early QML proposals were designed for speedup over classical ML algorithms.
The NISQ reality forced a different question.

Rather than: *"Can quantum computers speed up known ML algorithms?"*
Variational circuits ask:

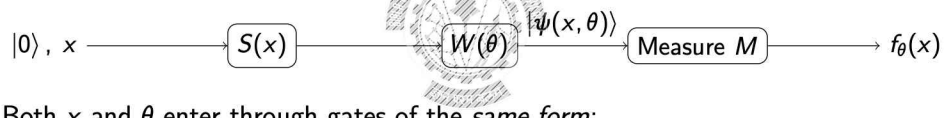
"Can quantum circuits define a genuinely new ML model family useful on near-term hardware?"

This shift in perspective opened new research questions:

- What *types of functions* do quantum models express?
- Can entanglement and superposition provide a practical advantage?
- How do we train these models reliably?

Anatomy of a Variational Quantum Circuit

$$U(x,\theta) = W(\theta)\, S(x)$$



Both x and θ enter through gates of the *same form*:

$$U(a) = e^{-iaG} \quad \text{(time evolution; } a \in \{x_i, \theta_k\}\text{)}$$

Data and parameters are architecturally interchangeable. They affect the expressibility and gradient computation.

The Classical Optimisation Loop - Scope Boundary

The quantum circuit is the **model**. Training is handled classically.

$$\underbrace{x \longrightarrow U(x,\theta) \longrightarrow M \longrightarrow \mathcal{L}(\theta)}_{\text{Quantum processor}} \longrightarrow \underbrace{\nabla_\theta \mathcal{L} \longrightarrow \theta_{\text{new}}}_{\text{Classical processor}}$$

Classical processor responsibilities:

- Evaluate the data-dependent loss $\mathcal{L}(\theta)$
- Compute parameter updates (gradient descent, Adam, RMSProp)
- Return updated θ to the quantum device

Module 6: *what the circuit computes* and *how gradients are obtained on the quantum side.*
Module 7: Full hybrid pipeline design - integrating quantum layers into PyTorch.

Measurement Interpretation → Model Type

After $U(x,\theta)\,|0\rangle = |\psi(x,\theta)\rangle$, the measurement strategy fixes the *type* of ML model:

Measurement	Output	Model type
Expectation value $\langle M \rangle$	Real number	Deterministic (regression/classification)
Outcome probability $p(y\|x)$	Label distribution	Supervised probabilistic
Sampled bit string $x \sim p_\theta$	Synthetic data	Generative / unsupervised

Table of Contents

Definition: Deterministic Quantum Model

Let $\mathcal{X}$ be a data input domain and $U(x,\theta)$ a quantum circuit depending on inputs $x \in \mathcal{X}$ and parameters $\theta \in \mathbb{R}^K$. Let $\mathcal{M}$ be a Hermitian observable. Denote by $|\psi(x,\theta)\rangle$ the state prepared by $U(x,\theta)|0\rangle$.

The **deterministic variational quantum model** is: $f_\theta(x) = \langle \psi(x,\theta)|\mathcal{M}|\psi(x,\theta)\rangle$
In density matrix notation: $f_\theta(x) = \mathrm{tr}\{\mathcal{M}\,\rho(x,\theta)\}$

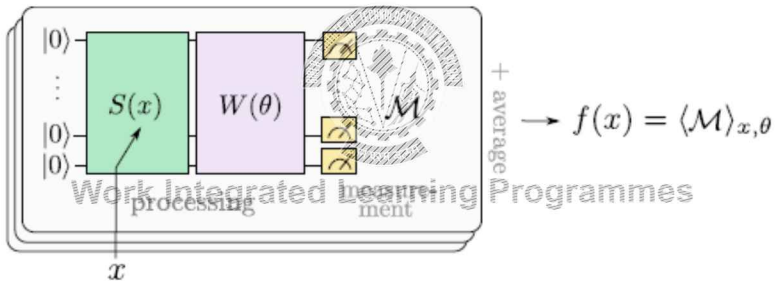
where $\rho(x,\theta) = U(x,\theta)^\dagger |0\rangle\langle 0| U(x,\theta)$.

We abbreviate this as $f_\theta(x) = \langle \mathcal{M} \rangle_{x,\theta}$.

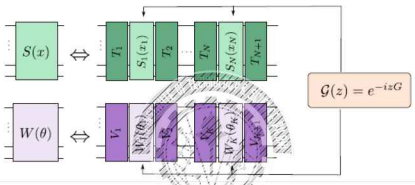
The circuit $U(x,\theta)$ can have any internal structure. A popular choice consists of a data embedding block $S(x)$ and a parametrised block $W(\theta)$:

$$U(x,\theta) = W(\theta)\,S(x)$$

Circuit Structure of $U(x, \theta)$



Gate Decomposition of $S(x)$ and $W(\theta)$



Both blocks decompose into single-feature/parameter gates and fixed unitaries T_i, V_k:

$$S(x) = T_{N+1} \prod_{i=1}^{N} S_i(x_i)\, T_i \qquad W(\theta) = V_{K+1} \prod_{k=1}^{K} W_k(\theta_k)\, V_k$$

Every gate has the **time-evolution form** $\mathcal{G}(a) = e^{-iaG}$, where G is the generator and $a \in \{x_i, \theta_k\}$.

Observations

Important observations:

1. The simple structure $U(x, \theta) = W(\theta)S(x)$ is only one possible architecture; practical quantum circuits may interleave encoding and trainable gates.
2. Data encoding can be repeated multiple times (data re-uploading), allowing the same input features to be injected into the circuit more than once.
3. The input features may first undergo classical preprocessing (feature maps) before being encoded into the quantum circuit.

Table of Contents

When an Expectation Value Is Not Enough

Deterministic models compress quantum information into a single number $\langle \mathcal{M} \rangle_{x,\theta}$. This is appropriate for supervised prediction tasks.

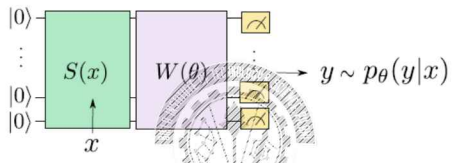
Many problems instead require modelling an entire distribution $p(x)$:

Task	What must be modelled
Image synthesis	$p(\text{image})$ over pixel configurations
Molecular design	$p(\text{molecule})$ over chemical structures
Density estimation	$p(x)$ from unlabelled data
Anomaly detection	$p(x)$ to flag low-probability events

Quantum circuits are **naturally probabilistic**. For an n-qubit state $|\psi\rangle = \sum_i a_i |i\rangle$, the Born rule gives:

$$p(i) = |a_i|^2, \qquad \sum_i p(i) = 1$$

Supervised Probabilistic Model: $p_\theta(y|x)$



Supervised probabilistic quantum model

A supervised probabilistic quantum model for a conditional distribution is defined by:

$$p_\theta(y|x) = |\langle y|\psi(x,\theta)\rangle|^2$$

where $|\psi(x,\theta)\rangle = U(x,\theta)|0\rangle$ and $\{|y\rangle\}$ are the eigenstates of $\mathcal{M}$ labelling outputs $y \in \mathcal{Y}$.

Supervised Probabilistic Model - Examples

Binary classification ($y = \pm 1$):
Map labels to eigenstates of σ^z: $y = +1 \leftrightarrow |0\rangle$ (eigenvalue $+1$), $y = -1 \leftrightarrow |1\rangle$ (eigenvalue -1).

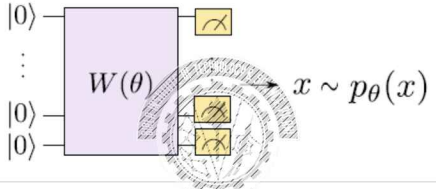
$$p_\theta(+1|x) = |\langle 0|\psi(x,\theta)\rangle|^2, \qquad p_\theta(-1|x) = |\langle 1|\psi(x,\theta)\rangle|^2$$

D-class classification ($D = 2^n$):
Use computational basis states $|d\rangle$, $d = 0, \ldots, D-1$, on n qubits to represent each class. One measurement shot directly samples a class label.

Outcome y	Amplitude a_y	$p_\theta(y|x) = |a_y|^2$	Class label
$\|00\rangle$	0.866	0.750	Class 0
$\|01\rangle$	0.354	0.125	Class 1
$\|10\rangle$	0.354	0.125	Class 2
$\|11\rangle$	0.000	0.000	Class 3
Total		1.000	

Prediction: $\hat{y} = \arg\max_y p_\theta(y|x) = |00\rangle$ (Class 0, probability 0.750).

Unsupervised Generative Model: $p_\theta(x)$



Definition 5.3 - Unsupervised probabilistic quantum model

Let $\mathcal{X}$ be an input domain, $W(\theta)$ a unitary with $|\psi(\theta)\rangle = W(\theta)|0\rangle$, and $\mathcal{M}$ a measurement in diagonal basis with outcomes $x \in \mathcal{X}$. An unsupervised probabilistic quantum model is defined by the distribution:

$$p(x) = |\langle x|\psi(\theta)\rangle|^2$$

Key Asymmetry: Sampling Easy, Density Hard

Sampling from p_θ	**Evaluating $p_\theta(x)$ for a specific x**
Easy - run circuit once per sample	Hard - may require exponentially many shots for large n
Hardware-native - one measurement gives one sample $x \sim p_\theta$	Number of shots to estimate $p_\theta(x)$ grows exponentially with n
Quantum circuits are naturally generative	It may in fact not be easy to compute or estimate on a quantum computer

Consequence for training

Maximum likelihood training requires evaluating $p_\theta(x^{(m)})$ for each training example $x^{(m)}$ - this is the *hard* part. Training strategies that work only with samples (not density values) are therefore strongly preferred.

Note: we can reinterpret $p_\theta(x) = |\langle x|\psi(\theta)\rangle|^2$ as a model function $f_\theta(x)$ with basis encoding and $\mathcal{M} = |\psi(\theta)\rangle\langle\psi(\theta)|$. Many insights from deterministic models apply here too.

Deterministic vs Probabilistic: Summary

Both model types use the same circuit $U(x, \theta)$. The difference is entirely in how measurement outcomes are used.

	Deterministic	**Probabilistic**
Output	$f_\theta(x) = \langle \mathcal{M} \rangle_{x,\theta} \in \mathbb{R}$	$p_\theta(y\|x)$ or $p_\theta(x)$
Measurement use	Average over shots	Distribution over outcomes
Tasks	Regression, classification	Generation, density estimation
Training signal	Prediction error	Distribution matching
Information used	Compressed into one number	Full distribution exploited

Unifying observation

$p_\theta(x) = |\langle x|\psi(\theta)\rangle|^2$ is itself a quantum expectation value, so the parameter-shift

Table of Contents

Single-Qubit Classifier Circuit

Circuit: $|0\rangle \xrightarrow{R_x(x)} \xrightarrow{\mathrm{Rot}(\theta_1,\theta_2,\theta_3)} \xrightarrow{\sigma_z}$

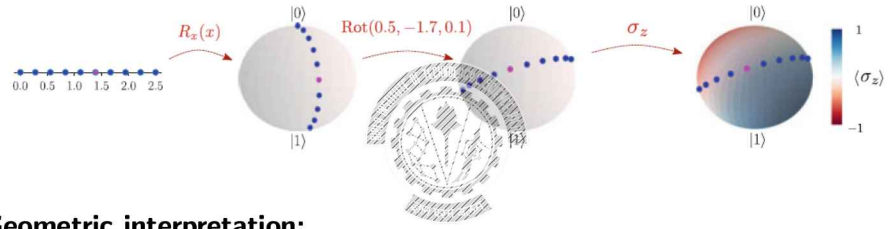
$R_x(x) = e^{-i\frac{x}{2}\sigma_x}$ encodes the input; $\mathrm{Rot}(\theta_1, \theta_2, \theta_3) = R_z(\theta_1)R_y(\theta_2)R_z(\theta_3)$ is the trainable ansatz.

After matrix multiplication: $\boxed{f_\theta(x) = \cos\theta_2 \cos x \; - \; \sin\theta_1 \sin\theta_2 \sin x}$

Observations:

- Output is a *weighted sum of* $\cos x$ and $\sin x$ - a trigonometric function, not a polynomial
- Only θ_1, θ_2 appear; θ_3 cancels in the σ_z expectation
- Frequencies present: $\{-1, 0, +1\}$ - set by eigenvalues of the R_x generator $\frac{\sigma_x}{2}$
- $f_\theta(x) \in [-1, +1]$ always - a valid bounded expectation value

Visualising the Model: Bloch Sphere



Geometric interpretation:

- $R_x(x)$ traces a circular arc on the Bloch sphere as x varies
- Rot is a rigid rotation of the entire sphere
- The measurement σ_z partitions the sphere into two hemispheres: north ($f_\theta > 0$, class $+1$) and south ($f_\theta < 0$, class -1)
- Training rotates the sphere so that class $+1$ and -1 data points land in the correct hemisphere

Single-Qubit Classifier Model

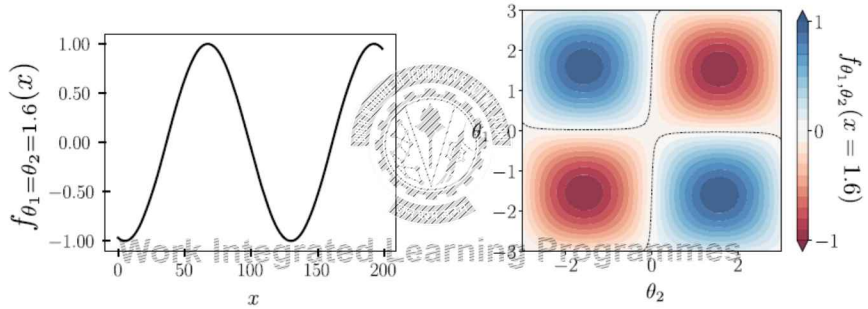
Model: $f_\theta(x) = \cos\theta_2 \cos x - \sin\theta_1 \sin\theta_2 \sin x$

Parameters: $\theta_1 = \theta_2 = \pi/4$:

$$f_\theta(x) = \tfrac{1}{\sqrt{2}} \cos x - \tfrac{1}{2} \sin x \approx 0.707 \cos x - 0.500 \sin x$$

x	$\cos x$	$\sin x$	$0.707 \cos x$	$-0.500 \sin x$	$f_\theta(x)$	$\hat{y}$
$\pi/4 \approx 0.785$	0.707	0.707	$+0.500$	-0.354	$+0.146$	$+\mathbf{1}$
$\pi/2 \approx 1.571$	0.000	1.000	$+0.000$	-0.500	-0.500	$-\mathbf{1}$
$\pi \approx 3.142$	-1.000	0.000	-0.707	$+0.000$	-0.707	$-\mathbf{1}$
$3\pi/2 \approx 4.712$	0.000	-1.000	$+0.000$	$+0.500$	$+0.500$	$+\mathbf{1}$

Model at $\theta_1 = \theta_2 = 1.6$: Function Plots



At $\theta_1 = \theta_2 = 1.6$: $\cos(1.6) \approx -0.029$, $\sin(1.6) \approx 0.9996$, so
$f_\theta(x) \approx -0.029 \cos x - 0.999 \sin x$

Model at $\theta_1 = \theta_2 = 1.6$: Function Plots

At $\theta_1 = \theta_2 = 1.6$: $\cos(1.6) \approx -0.029$, $\sin(1.6) \approx 0.9996$, so

$$f_\theta(x) \approx -0.029 \cos x - 0.999 \sin x$$

The model is dominated by $-\sin x$; the cosine term is negligible. Decision boundary ($f_\theta = 0$) lies between $x = 2.5$ (class -1) and $x = 5.0$ (class $+1$).

$f_\theta(x) = \cos\theta_2 \cos x - \sin\theta_1 \sin\theta_2 \sin x$ is a *product of sines and cosines* in both x and the parameters:
This Limits which functions the model can represent

Evaluating the Model

With $\mathcal{M} = \sum_i \mu_i |\mu_i\rangle\langle\mu_i|$ in its eigenbasis:

$$f_\theta(x) = \langle\psi(x,\theta)|\mathcal{M}|\psi(x,\theta)\rangle = \sum_i \mu_i \underbrace{|\langle\mu_i|\psi(x,\theta)\rangle|^2}_{p(\mu_i)}$$

For $\mathcal{M} = \sigma_z$: $f_\theta(x) = p(0) - p(1)$

Practical estimation - run the circuit S times and average:

$$\hat{f}(x) = \frac{1}{S}\sum_{s=1}^{S} m^{(s)}, \qquad S = O(\epsilon^{-2}) \text{ shots for error } \epsilon$$

Important: measurement collapses the state - each shot requires a fresh circuit run from $|0\rangle$.

Shot Estimation of $\langle\sigma_z\rangle$

State: $|\psi\rangle = \sqrt{0.7}|0\rangle + \sqrt{0.3}|1\rangle$

True value: $\langle\sigma_z\rangle = p(0) - p(1) = 0.7 - 0.3 = +\mathbf{0.4}$

10-shot example: $+1, +1, -1, +1, +1, -1, +1, +1, -1, +1 \Rightarrow 7(+1)+3(-1)$

$$\hat{f} = \frac{7-3}{10} = +0.4 \quad \checkmark$$

Shots S	Error $\sim 1/\sqrt{S}$	Estimate
10	± 0.316	$+0.4 \pm 0.3$ (unreliable)
100	± 0.100	$+0.4 \pm 0.1$
400	± 0.050	$+0.4 \pm 0.05$
1000	± 0.032	$+0.4 \pm 0.03$

From Expectation Value to Prediction

Output $f_\theta(x) = \langle\mathcal{M}\rangle_{x,\theta} \in [-1, +1]$ for $\mathcal{M} = \sigma_z$.

For regression: use $f_\theta(x)$ directly. $\mathcal{L}(\theta) = \frac{1}{N}\sum_{m=1}^{N}\big(y_m - f_\theta(x_m)\big)^2$
For binary classification: threshold at zero.

$$\hat{y} = \begin{cases} +1 & f_\theta(x) \geq 0 \\ -1 & f_\theta(x) < 0 \end{cases}$$

The expectation value acts like the output neuron of a classical binary classifier.

Extending to Multiple Qubits (Entanglement)

For n features $x = (x_1, \ldots, x_n)$, use n qubits with $S(x) = \bigotimes_{i=1}^{n} R_x(x_i)$ followed by an entangling ansatz $W(\theta)$.

Without entanglement:
$f_\theta(x) = \sum_{i=1}^{n} g_i(x_i)$
Features interact independently. Additive model only — cannot capture x_i-x_j correlations. Cannot learn XOR-type boundaries.

With CNOT entangling layers:
3-qubit R_x encoding + $R_y(\theta)$ layers + CNOT ring, measure $\langle Z\rangle$
Cross-frequency terms $\cos(x_i \pm x_j)$ appear.
Multivariate decision boundaries become accessible.

Without Entanglement, a multi-qubit VQC is just n independent single-qubit classifiers. Entangling gates are the mechanism by which different features interact.

Where Does the Learning Capacity Come From?

The expressive power of $f_\theta(x) = \langle\psi(x,\theta)|\mathcal{M}|\psi(x,\theta)\rangle$ is jointly determined by:

Factor	Effect
Data encoding $S(x)$	Sets the *frequency spectrum* of f_θ; richer encoding ⇒ richer function class
Generator G	Eigenvalues of G determine which frequencies are accessible
Entangling gates	Enable cross-feature correlations; product-state circuits produce only additive models
Circuit depth	Repeated encoding layers extend the Fourier spectrum
Parameters θ	Control Fourier coefficients $c_\omega(\theta)$; training selects a specific function

Encoding determines *what functions are reachable*; training *selects one*.

The VQC as a Kernel Method: A Preview

Data encoding $S(x)$ defines a quantum feature map $x \mapsto \rho(x) = S(x)|0\rangle\langle 0|S^\dagger(x)$.
The overlap between two encoded states defines a **quantum kernel**:

$$\kappa(x,x') = \mathrm{Tr}\big(\rho(x)\,\rho(x')\big) = |\langle\psi(x)|\psi(x')\rangle|^2$$

The optimal VQC classifier is an SVM with this kernel:

- $W(\theta)$ learns the optimal measurement basis
- $S(x)$ determines the kernel and the achievable decision boundaries

Key insight

The encoding $S(x)$ is the most critical design decision in a VQC. Training can only select among functions already accessible via the chosen encoding.

This kernel connection is developed fully in Chapter 6 (Quantum Models as Kernel Methods).

Table of Contents

The Born Machine

Remove the encoding block $S(x)$. Let $W(\theta)$ alone prepare the state:

$$|\psi(\theta)\rangle = W(\theta)|0\rangle$$

The circuit implements the probabilistic model (Definition 5.3):

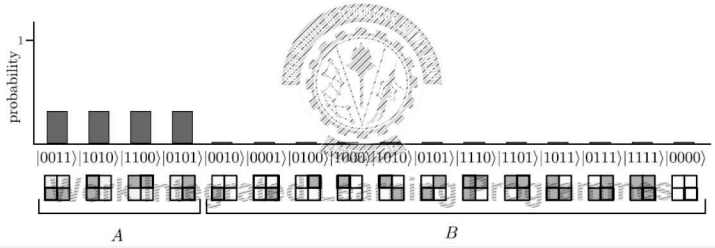
$$p(x) = |\langle x|\psi(\theta)\rangle|^2, \qquad x \in \{0,1\}^n$$

Each measurement shot produces one synthetic sample $x \sim p_\theta$. Training adjusts θ so that $p_\theta \approx p_{\text{data}}$.

Born machines are also known as *Quantum Circuit Born machines* (QCBMs) - named after the Born rule and Boltzmann machines.

Textbook Example: Bars-and-Stripes

Task: Learn p_{data} uniform over the 4 bars-and-stripes patterns of 2×2 binary images.



4 pixel-qubits (visible) encode each image via basis encoding. 3 additional qubits are **hidden** (never measured). Circuit: $W(\theta)|0\rangle^{\otimes 7}$, measure only the 4 visible qubits.

The Optimal State and the Role of Hidden Qubits

Optimal state (no hidden units):

$$|\psi\rangle = \tfrac{1}{2}\big(|1010\rangle + |0101\rangle + |1100\rangle + |0011\rangle\big)$$

This gives equal probability $\frac{1}{4}$ to each bars-and-stripes pattern and zero to all others.

Why add hidden qubits?

The 4 visible qubits represent the image we want to generate. The additional 3 hidden qubits participate in the computation but are **never measured**. They interact with the visible qubits through the variational circuit

$$|\psi(\theta)\rangle = W(\theta)|0000000\rangle.$$

At the end, only the visible qubits are measured. Ignoring the hidden qubits leaves the visible register in a *mixed state*, which can represent richer probability distributions than a pure 4-qubit state.

Numerical Example: Born Machine Probability Table

Setup: 2-qubit Born machine. Before training ($\theta = 0$): $W(0)|00\rangle = |00\rangle$, so all probability is on $|00\rangle$.

x	$p_{\text{data}}(x)$	$p_\theta(x)$ before	$p_\theta(x)$ after	D_{KL} contribution (after)
$\|00\rangle$	0.50	1.00	0.49	$0.50 \log \frac{0.50}{0.49} \approx +0.010$
$\|01\rangle$	0.25	0.00	0.26	$0.25 \log \frac{0.25}{0.26} \approx -0.010$
$\|10\rangle$	0.15	0.00	0.15	$0.15 \log \frac{0.15}{0.15} = 0.000$
$\|11\rangle$	0.10	0.00	0.10	$0.10 \log \frac{0.10}{0.10} = 0.000$
Total	1.00	1.00	1.00	$D_{KL} \approx 0.000$

Training a Born Machine: Distribution Matching

Objective: minimise $D_{KL}(p_{\text{data}} \| p_\theta)$:

$$\mathcal{L}(\theta) = D_{KL}(p_{\text{data}} \| p_\theta) = \sum_x p_{\text{data}}(x) \log \frac{p_{\text{data}}(x)}{p_\theta(x)}$$

The fundamental asymmetry:

Sampling from p_θ	**Evaluating $p_\theta(x)$ for a specific x**
Easy - one circuit run per sample	Hard - requires exponentially many shots for large n
Hardware-native	May in fact not be easy to compute or estimate on a quantum computer

Generative Adversarial Training for Born Machines

When direct density evaluation is too costly, a **quantum GAN** bypasses it entirely.

Setup:

- **Generator** $G(\theta)$: Born machine preparing $p_\theta(x)$
- **Discriminator** $D(\phi)$: distinguishes real samples vs. generated samples

Adversarial objectives:

$$\mathcal{L}_D(\phi) = p(1|D_{\text{real}}) - p(1|D_{\text{gen}}) \qquad \text{(discriminator: separate real from fake)}$$

$$\mathcal{L}_G(\theta) = -p(1|G) \qquad \text{(generator: fool the discriminator)}$$

D and G are trained in alternation. At convergence: $p_\theta \approx p_{\text{data}}$.

G and D don't evaluate $p_\theta(x)$, they work with samples from p_θ and p_{data}. This makes the training *procedure* well-defined on hardware.

Quantum GANs in Practice: Open Challenges

Challenge	What happens in practice
Barren plateaus (doubled)	Both $G(\theta)$ and $D(\phi)$ independently suffer *McClean et al. (2018); Rudolph et al.(2024)*
Mode collapse	Generator concentrates on a few outputs; impossible to detect directly since $p_\theta(x)$ cannot be inspected. *VAE-QWGAN (2025)*
GAN instability	Zero-sum training causes convergence failure and slow training even before quantum noise is considered. *Rudolph et al.(2024)*
Scalability	Best results require classical compression first (latent-space QGANs); quantum advantage at scale remains unclear. *Chang et al. arxiv (2024)*

Table of Contents

Intuition: The Music Analogy

An orchestra:

- Each instrument plays at a *fixed pitch* (frequency)
- The composer decides which instruments play and how loudly each sounds
- The available pitches are fixed by which instruments are in the orchestra
- A richer orchestra enables richer music

A variational quantum circuit:

- Each encoding gate contributes *fixed oscillation rates* (frequencies)
- Training decides how strongly each rate appears (adjusts the coefficients)
- The available rates are fixed by the encoding generator G - before training begins
- A richer encoding enables a richer function class

	Orchestra	VQC
What is fixed	Set of available pitches	Frequency spectrum Ω

Starting from What We Already Know

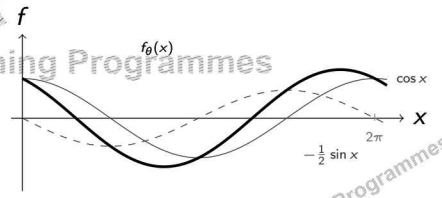
From the single-qubit VQC we derived today:

$$f_\theta(x) = \underbrace{\cos\theta_2}_{c_1}\cos x \; - \; \underbrace{\sin\theta_1 \sin\theta_2}_{c_2}\sin x$$

This is a **weighted sum of two sinusoids**: one $\cos x$ and one $\sin x$.

What training can do:

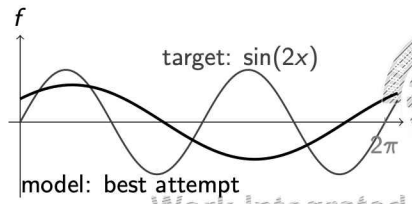
Adjust θ_1, θ_2 to change the weights c_1 and c_2 - shift and scale the sinusoidal components.



What training cannot do:

Access $\cos(2x)$, $\sin(3x)$, or any other oscillation rate.

The Spectrum Limits What the Model Can Fit



Our single-R_x VQC **cannot fit** $\sin(2x)$ because frequency 2 is simply not available in its function class.

No matter how long we train, or how many parameters we use, the circuit can never reproduce a function that oscillates twice per period.

Expressibility is an *encoding* problem, not a *training* problem.
If the data requires frequency 2, we must change the encoding - not train longer.

Table of Contents

Practice Problem

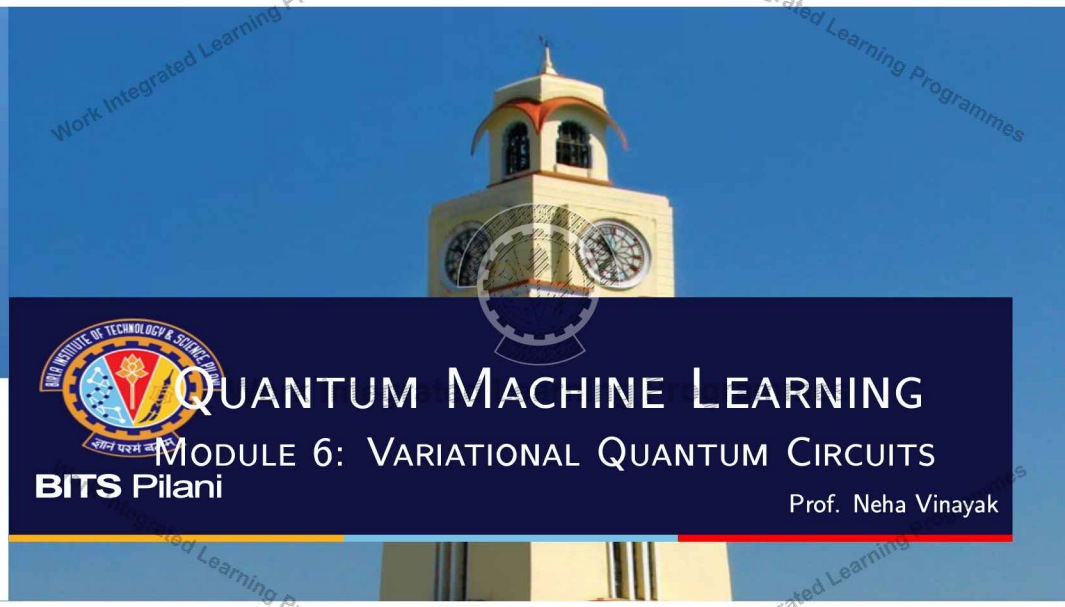
A trained 2-qubit circuit produces:

$$p(00) = 0.25, \quad p(01) = 0.375, \quad p(10) = 0.375, \quad p(11) = 0.$$

1. Interpret the circuit as a **deterministic model** with $M = \sigma_z \otimes I$. Using $f_\theta(x) = p(0\cdot) - p(1\cdot)$, determine the predicted class.
2. Interpret the same circuit as a **4-class probabilistic model**. Which class is predicted?
3. Why can the deterministic and probabilistic models give different predictions even though they use the same quantum state?
4. A student removes all CNOT gates and the model can no longer learn correlated features. Explain why entanglement is necessary.
5. How would you convert this circuit into a **Born machine**? State the changes in (i) circuit structure, (ii) output, and (iii) learning objective.

BITS Pilani

Thank you :)



Quantum Machine Learning

Module 6: Variational Quantum Circuits

BITS Pilani

Prof. Neha Vinayak

Table of Contents

Starting from What We Already Know

In Session 9 we derived:

$$f_\theta(x) = \cos\theta_2 \cos x \; - \; \sin\theta_1 \sin\theta_2 \sin x$$

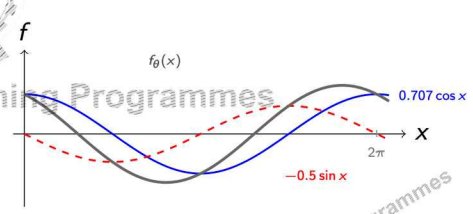
This is a **weighted sum of two sinusoids**: one $\cos x$ term and one $\sin x$ term.

What training can do:

Adjust θ_1, θ_2 to change the *weights*:
$c_1 = \cos\theta_2, \quad c_2 = -\sin\theta_1 \sin\theta_2$

What training cannot do:

Access $\cos(2x)$, $\sin(3x)$, or any other oscillation rate. The *available rates* are fixed before training begins.



Question: Is this specific to our circuit, or does every VQC produce a sum of sinusoids?

The Music Analogy

An orchestra:

- Each instrument plays a *fixed note* (frequency)
- The composer decides which instruments play and how loudly
- The available notes are fixed by which instruments are present
- Richer orchestra - richer composition

A variational quantum circuit:

- Each encoding gate contributes *fixed oscillation rates* (frequencies)
- Training decides how strongly each rate appears (coefficients)
- The available rates are fixed by the encoding generator G
- Richer encoding - richer function class

	Orchestra	**VQC**
What is fixed	Set of notes	Frequency spectrum Ω (by G)
What is learned	Volume of each note	Coefficient of each sinusoid

Function Class of Quantum Models

Theorem (Function Class of a VQC)

Data encoding: $S(x) = e^{-ixG}$, where G has eigenvalues $\{\lambda_1, \ldots, \lambda_d\}$ Trainable blocks $W(\theta)$
Model Output can be written as:

$$f_\theta(x) = \sum_{\omega \in \Omega} c_\omega(\theta) e^{i\omega x} = \sum_{\omega > 0} [a_\omega(\theta)\cos(\omega x) + b_\omega(\theta)\sin(\omega x)] + c_0(\theta),$$

where $\Omega = \{\lambda_s - \lambda_t \mid s, t\}$
is the set of frequencies determined by the encoding generator G.

Every VQC output is a weighted sum of sinusoids. The available sinusoids are fixed by the encoding generator G.

Two Key Consequences

1: Encoding controls the frequency set

The spectrum Ω is *completely determined by the encoding generator* G. It does not depend on the ansatz $W(\theta)$ or on training.

2: Training controls the coefficients

The weights $a_\omega(\theta)$ and $b_\omega(\theta)$ depend on θ. Training adjusts these - but *cannot introduce new frequencies.*

What encoding $S(x)$ fixes	What ansatz $W(\theta)$ controls
Which sinusoids are available	How much of each sinusoid appears
The *shape vocabulary* of the model	The *specific shape* learned
Set once, before training	Adjusted during training

Finding Ω for Pauli R_x Encoding

Encoding gate: $R_x(x) = e^{-i\frac{x}{2}\sigma_x}$, generator $G = \frac{\sigma_x}{2}$

Step 1 - Eigenvalues of $G = \frac{\sigma_x}{2}$:

$$\frac{\sigma_x}{2} = \frac{1}{2}\begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} \Rightarrow \lambda_1 = +\tfrac{1}{2}, \quad \lambda_2 = -\tfrac{1}{2}$$

Step 2 - All pairwise differences $\lambda_s - \lambda_t$: $\Omega = \{-1, 0, +1\}$

λ_s	λ_t	$\lambda_s - \lambda_t$	Frequency
$+\frac{1}{2}$	$+\frac{1}{2}$	0	$\omega = 0$
$+\frac{1}{2}$	$-\frac{1}{2}$	$+1$	$\omega = +1$
$-\frac{1}{2}$	$+\frac{1}{2}$	-1	$\omega = -1$
$-\frac{1}{2}$	$-\frac{1}{2}$	0	$\omega = 0$

Confirming the Model Form

From $\Omega = \{-1,\ 0,\ +1\}$, Theorem gives:

$$f_\theta(x) = a_0(\theta) + a_1(\theta)\cos x + b_1(\theta)\sin x$$

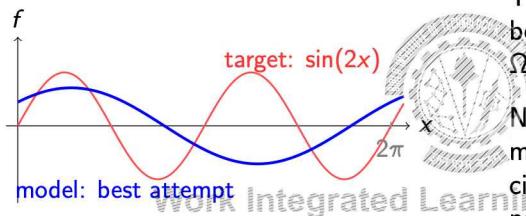
Check against the Session 9 result:

$$f_\theta(x) = \underbrace{\cos\theta_2}_{a_1}\cos x \ - \ \underbrace{\sin\theta_1 \sin\theta_2}_{-b_1}\sin x \qquad (a_0 = 0 \text{ for this circuit})$$

What this tells us about the model:

- Only one oscillation rate ($\omega = 1$) is available
- No amount of training can make the model fit $\cos(2x)$ or $\sin(3x)$
- This is not a weakness of the particular θ values - it is a structural limit

The Spectrum Limits What the Model Can Fit



The model **cannot fit** $\sin(2x)$ because frequency 2 is not in $\Omega = \{-1, 0, +1\}$.

No matter how long we train or how many parameters θ we use, the circuit with a single R_x encoding can never reproduce a function that oscillates twice per period.

Expressibility is an *encoding problem*, not a *training problem*. If the data requires frequency 2, we must change the encoding - not train longer.

Enriching the Spectrum: Repeating the Encoding

Repeat the encoding block L times inside the circuit:

$$U(x,\theta) = W_{L+1}(\theta_{L+1}) \prod_{l=1}^{L} S(x)\, W_l(\theta_l)$$

Each repetition *adds* frequency combinations. For Pauli encoding:

L	Spectrum Ω	Model form
1	$\{-1,\ 0,\ +1\}$	$a_0 + a_1\cos x + b_1\sin x$
2	$\{-2,\ -1,\ 0,\ +1,\ +2\}$	adds $a_2\cos 2x + b_2\sin 2x$
3	$\{-3, \ldots, 0, \ldots, +3\}$	adds $a_3\cos 3x + b_3\sin 3x$
L	$\{-L, \ldots, 0, \ldots, +L\}$	$L+1$ sinusoidal components

Towards a Universal Function Approximator

As $L \to \infty$, the spectrum covers all integer frequencies and the model becomes a universal function approximator - it can represent *any* smooth periodic function to arbitrary precision.

Important: A rich spectrum does not automatically mean a rich model. Whether all coefficients c_n can be *independently controlled* depends on the ansatz $W(\theta)$. A deep encoding with a weak ansatz still produces a limited model.

The architectural strategy of repeating $S(x)$ is called **data re-uploading**. The design of such circuits is covered in **Module 7**.

Periodicity: A Practical Preprocessing Consequence

Since all frequencies ω are integer-valued for Pauli encoding:

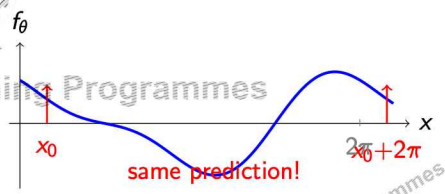
$$f_\theta(x + 2\pi) = f_\theta(x) \quad \text{for all } x$$

The circuit cannot distinguish x from $x + 2\pi$.

Practical consequence:
Consider a fraud detection VQC.
Transaction amount $x \in [0, 10000]$.
Without rescaling:

- \$100 and \$106.28 $(= 100 + 2\pi)$ give *identical* predictions
- Periodic artefacts corrupt the model

Always scale inputs to $[0, 2\pi)$ before encoding.



The Same Structure Holds in Parameter Space

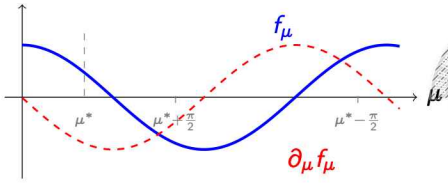
Theorem 5.1 described $f_\theta(x)$ as sinusoidal in the *inputs* x.
The same derivation applies to the *parameters* θ. For a single Pauli-generated gate $G(\mu) = e^{-i\mu G}$ with $G^2 = I$:

$$G(\mu) = \cos\mu \cdot I - i \sin\mu \cdot G \quad \Rightarrow \quad f_\mu = \alpha + \beta\cos\mu + \gamma\sin\mu$$

f_μ is a sinusoid in the parameter μ. Its derivative is another sinusoid:

$$\partial_\mu f_\mu = -\beta\sin\mu + \gamma\cos\mu$$

Why the Parameter-Side Structure Matters



Since f_μ is sinusoidal in μ:

$$\partial_\mu f_\mu = \frac{f_{\mu+\frac{\pi}{2}} - f_{\mu-\frac{\pi}{2}}}{2}$$

The gradient at μ^* can be computed by evaluating the *same circuit* at two shifted parameter values.

No approximation. Exact.

This is the **parameter-shift rule**

Expressibility → sinusoidal structure → exact gradients.

Expressibility: Summary

What we established:

1. Every VQC output is a weighted sum of sinusoids
2. Frequencies $\Omega = \{\lambda_s - \lambda_t\}$ fixed by encoding generator
3. Training adjusts coefficients – not frequencies
4. Repeating $S(x)$ extends Ω; $L \to \infty$ gives universal approximation
5. Input features must be scaled to $[0, 2\pi)$
6. Sinusoidal structure in θ enables exact gradient computation

$$S(x) \longrightarrow \Omega \text{ (which sinusoids)}$$
$$W(\theta) \longrightarrow c_\omega(\theta) \text{ (how much of each)}$$

Table of Contents

The Gradient We Need

Training updates parameters by gradient descent:

$$\theta_k \leftarrow \theta_k - \eta \frac{\partial \mathcal{L}}{\partial \theta_k}, \qquad k = 1, \ldots, K$$

For a VQC with K parameters, we need K partial derivatives at each training step.

Classical neural network	Variational quantum circuit
Backpropagation: all K derivatives in *one backward pass*	Each derivative requires *separate circuit runs*
Intermediate activations stored and reused	No-cloning: cannot copy or store quantum states
Cost: $O(1)$ model evaluations	Cost: at least $O(K)$ circuit evaluations

Problem: Derivative $\neq$ Quantum Expectation

Write the circuit as $V\,G(\mu)\,W$ with $|\psi\rangle = W\,|0\rangle$, $B = V^\dagger M V$. The model: $f_\mu = \langle\psi|\,G^\dagger(\mu)\,B\,G(\mu)\,|\psi\rangle$.

Applying the product rule:

$$\partial_\mu f_\mu = \langle\psi|\,G^\dagger(\mu)\,B\,(\partial_\mu G)\,|\psi\rangle + \langle\psi|\,(\partial_\mu G)^\dagger\,B\,G(\mu)\,|\psi\rangle$$

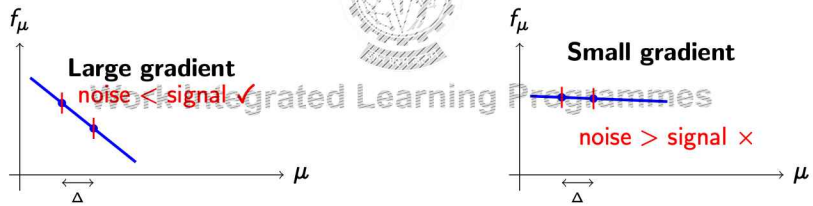
Two problems with each term:

- Bra state $\langle\psi|\,G^\dagger$ and ket state $(\partial_\mu G)\,|\psi\rangle$ are *different* — this is not a valid expectation value $\langle\cdot|M|\cdot\rangle$
- $\partial_\mu G(\mu)$ is generally *not unitary* - it cannot be run as a quantum gate

Consequence: The gradient of a quantum model cannot be obtained by running the same circuit and reading off a measurement. A new approach is needed.

The Naive Fix: Finite Differences

Avoid the derivative problem by approximating it: $\partial_\mu f_\mu \approx \frac{f_{\mu+\Delta} - f_\mu}{\Delta}$
Two circuit runs. Simple. Works classically. **Fails on quantum hardware**: each f is estimated from shots.



Same absolute shot noise; smaller gradient makes the signal undetectable.

Example: Why Finite Differences Fail

Model: $f(\mu) = \cos\mu$. **True gradient at** $\mu = \pi/2$: $\partial_\mu f = -1$.

With $S = 100$ shots, measurement noise $\approx 1/\sqrt{100} = 0.1$; noise on the difference $\approx \sqrt{2} \times 0.1 \approx 0.14$.

Δ	Signal	Noise	Gradient estimate	
0.100	0.0998	± 0.14	-1.0 ± 1.4	(noisy)
0.010	0.0100	± 0.14	-1.0 ± 14	**(useless)**
0.001	0.0010	± 0.14	-1.0 ± 140	**(useless)**

The dilemma: Smaller Δ gives a better mathematical approximation of the derivative, but the numerator $f(\mu + \Delta) - f(\mu)$ shrinks proportionally while shot noise stays constant at ± 0.14. Eventually noise completely dominates the signal. Shots $S \propto \Delta^{-2}$. As $\Delta \to 0$, the required shot count diverges. **There is no good value of Δ on quantum hardware.**

What Should an Ideal Quantum Gradient Method Do?

Finite differences are not suitable for training VQCs on quantum hardware. An ideal quantum gradient method must satisfy three requirements simultaneously.

	Requirement	Why it is necessary
✓	**Exact**	Approximation error compounds over K parameters and many training steps; systematic bias does not average away
✓	**Quantum-executable**	Must be expressible as quantum expectation values so it can be estimated with $O(\varepsilon^{-2})$ shot machinery
✓	**Efficient**	Must require only a small fixed number of circuit evaluations per parameter; finite differences need $S \propto \Delta^{-2}$ shots

Can such a method exist? **Yes - the parameter-shift rule.**

The Bridge to the Parameter-Shift Rule

The answer comes from the sinusoidal structure proved in Section 1.

For every trainable parameter μ, the circuit output is: $f(\mu) = A + B\cos\mu + C\sin\mu$ $f(\mu)$ is **exactly sinusoidal** in μ. Its derivative $\partial_\mu f = -B\sin\mu + C\cos\mu$ is also a sinusoid with the same B, C - also a valid quantum expectation value.

Can we express $\partial_\mu f$ using only evaluations of the *original unmodified circuit* at two shifted parameter values $f(\mu + s_1)$ and $f(\mu + s_2)$?

If yes:

- Gradient is **exact** - no $\Delta \to 0$ limit needed
- Each evaluation is a **quantum expectation value** - estimable from shots
- Only **two circuit runs** per parameter - efficient

Yes - this is the parameter-shift rule.

Table of Contents

The Key Idea: Derivative as a Difference of Evaluations

Start with $f(\theta) = \cos\theta$. True gradient: $\partial_\theta f = -\sin\theta$.

Apply two standard trigonometric identities:

$$\cos\left(\theta + \tfrac{\pi}{2}\right) = -\sin\theta \quad \text{and} \quad \cos\left(\theta - \tfrac{\pi}{2}\right) = +\sin\theta$$

Subtract and divide by 2:

$$\boxed{\frac{df}{d\theta} = -\sin\theta = \frac{f\left(\theta + \frac{\pi}{2}\right) - f\left(\theta - \frac{\pi}{2}\right)}{2}}$$

The derivative equals a difference of two function evaluations. No symbolic differentiation. No small Δ. Exact.

This works because $f(\theta) = \cos\theta$ is sinusoidal. Is every VQC output sinusoidal in its parameters?

The Condition: When Is f_μ Sinusoidal in μ?

For a circuit $U = V\,G(\mu)\,W$ where $G(\mu) = e^{-i\mu G}$:

The key condition: G is a Pauli operator satisfying $G^2 = I$. Then: $G(\mu) = \cos(\mu)\,I - i\sin(\mu)\,G$.

Standard rotation gates:

Gate	Write as $e^{-i\mu G}$ with	$G^2 = I$?	Shift rule?
$R_x(\theta)$	$\mu = \theta/2,\ G = \sigma_x$	✓	✓
$R_y(\theta)$	$\mu = \theta/2,\ G = \sigma_y$	✓	✓
$R_z(\theta)$	$\mu = \theta/2,\ G = \sigma_z$	✓	✓
Tensor products	$G = \sigma_i \otimes \sigma_j$	✓	✓

Any parametrised unitary can be decomposed into such building blocks.

Derivation: Why the Shift Rule Is Exact

With $G^2 = I$, write $|\psi\rangle = W|0\rangle$, $B = V^\dagger M V$. The model: $f_\mu = \langle\psi|\,G^\dagger(\mu)\,B\,G(\mu)\,|\psi\rangle$. Substituting $G(\mu) = \cos\mu \cdot I - i\sin\mu \cdot G$, the textbook shows:

$$K(\mu) = G^\dagger(\mu)\,B\,G(\mu) = A + B\cos\mu + C\sin\mu$$

Therefore:

$$f_\mu = \tilde{\alpha} + \tilde{\beta}\cos\mu + \tilde{\gamma}\sin\mu$$

f_μ is *exactly sinusoidal in* μ. Now use the identity:

$$\frac{d}{d\mu}\cos\mu = \frac{\cos(\mu+s) - \cos(\mu-s)}{2\sin s} \quad \text{(exact, for any } s \neq k\pi\text{)}$$

Applying to f_μ:

$$\partial_\mu f_\mu = \frac{f_{\mu+s} - f_{\mu-s}}{2\sin s}$$

Exact at every step. No approximation.

Definition 5.4: Parameter-Shift Rule

Definition 5.4 (Schuld & Petruccione, 2021)

A **parameter-shift rule** is an identity of the form: $\partial_\mu f_\mu = \sum_i a_i\, f_{\mu+s_i}$ For Pauli-generated gates with $G^2 = I$:

$$\boxed{\partial_\mu f_\mu = \frac{f_{\mu+s} - f_{\mu-s}}{2\sin s} \xrightarrow{s=\pi/2} \frac{f_{\mu+\pi/2} - f_{\mu-\pi/2}}{2}}$$

Why $s = \pi/2$? The formula holds for any $s \neq k\pi$. Choosing $s = \pi/2$ gives $\sin(\pi/2) = 1$, simplifying the denominator to 2.

Method	Gradient quality	Shift size
Finite difference	Approximate ($O(\Delta)$ error)	Small $\Delta \to 0$ (noise-sensitive)
Parameter-shift	**Exact**	Fixed $\pi/2$ (noise-robust)

Worked Example: Full Gradient Computation

Circuit: $|0\rangle \xrightarrow{R_y(\theta)} \sigma_z$ **Model:** $f(\theta) = \cos\theta$ **Evaluate at:** $\theta = \pi/3$

True gradient: $\partial_\theta f\big|_{\pi/3} = -\sin(\pi/3) = -\frac{\sqrt{3}}{2} \approx -0.866$

Step 1 - Run at $\theta + \pi/2 = 5\pi/6$: $f(5\pi/6) = \cos(5\pi/6) = -\sqrt{3}/2 \approx -0.866$
Step 2 - Run at $\theta - \pi/2 = -\pi/6$: $f(-\pi/6) = \cos(-\pi/6) = +\sqrt{3}/2 \approx +0.866$
Step 3 - Apply the shift rule:

$$\partial_\theta f = \frac{-0.866 - 0.866}{2} = \frac{-1.732}{2} = -0.866 \checkmark$$

Exact match. Two circuit evaluations. No approximation.
The shift $\pi/2$ is large, so both evaluations are at well-separated points - shot noise on each evaluation does not cancel the signal.

Cost of Computing the Full Gradient

One parameter: 2 circuit runs.
Full gradient (K parameters): $2K$ circuit runs, $2KS$ total shots.

Example: $K = 100$, $S = 10$ shots: $2 \times 100 \times 10 = 2000$ shots per step. At $1\,\mu$s per shot: 2 ms per gradient step.

Key property: The parameter-shift estimator is *unbiased* - S can be as small as 1 in early training.

	Classical	Param-shift
Runs for ∇f	$O(1)$	$2K$
Gradient	Exact	Exact
Memory	$O(\text{depth})$	None
On hardware	No	Yes

Extensions of the Parameter-Shift Rule

Multiple parameters:

$$\partial_\mu \mathcal{L}(\theta) = \sum_{l=1}^{j} \frac{f_{\theta,\mu_l+\pi/2} - f_{\theta,\mu_l-\pi/2}}{2}$$

Born machines: The same rule applies to $p_\theta(x) = |\langle x|\psi(\theta)|x|\psi(\theta)\rangle|^2$:

$$\partial_\mu p_\theta(x) = \frac{p_{\mu+\pi/2}(x) - p_{\mu-\pi/2}(x)}{2}$$

Higher derivatives: Apply the rule twice to get the Hessian ($4K^2$ runs).

Looking ahead - Section 5.3.3: The parameter-shift rule gives us *exact* gradients. But what if the exact gradient is exponentially small? With n qubits, the variance of $\partial_\mu f$ can scale as $O(2^{-n})$. Even exact computation of an exponentially small number requires exponentially many shots. This is the **barren plateau problem**.

Table of Contents

A Tempting Intuition - and Why It Fails

Deeper circuits enrich Ω and increase expressibility.

Deeper circuit $\Rightarrow$ richer model $\Rightarrow$ better performance

Practical experience says otherwise. Many highly expressive VQCs become *impossible to train*:

- Gradient descent takes thousands of steps with essentially no progress
- All parameter updates are numerically negligible
- The circuit can represent the answer but the optimiser cannot find it

The parameter-shift rule gives us *exact* gradients - but what if the exact gradient at almost every point in parameter space is 10^{-8}?

This is the barren plateau problem.

What Is a Barren Plateau?

Barren Plateau

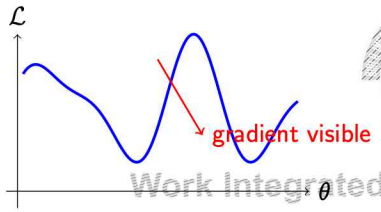
A barren plateau is a region of parameter space in which the gradients are, *with high probability*, close to zero in all their elements. For a family of models $f_{\theta,W}$ where the ansatz W is drawn from a distribution, if we sample W^* and θ^*, then the gradient $\nabla_\theta f_{\theta^*,W^*}$ is likely to vanish.

Two things to note carefully:

1. *"With high probability"* means this is a *statistical* statement about a family of circuits - not every circuit has a barren plateau, but most randomly initialised ones do
2. It is not a local flat region like a saddle point - the landscape is flat *almost everywhere*, so random restarts do not help

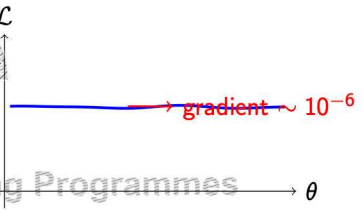
Visualising the Landscape

Useful landscape (few qubits):



Optimiser can follow the gradient downhill.

Barren plateau (many qubits):



Optimiser cannot distinguish any direction.

Same circuit structure; the landscape flattens as n grows.

Why Does This Happen? The Hilbert Space Argument

Imagine a vector in an **exponentially large space** (2^n dimensions).
We rotate it by a small angle φ with one gate $G(\mu)$, then apply *many random rotations* in all directions.
On average, the effect of φ on the final expectation becomes *negligible* - the single rotation is diluted across 2^n dimensions.

Moving a single step in a 2^n-dimensional space barely changes your position.

Fourier coefficient view:

A barren plateau means the ansatz $W(\theta)$ is stuck in a subspace where the Fourier coefficients $c_\omega(\theta)$ are nearly constant. Training cannot effectively vary them.

The same expressibility that makes the circuit powerful (large Hilbert space, many entangling gates) is exactly what causes the gradient to vanish.

How Fast Do Gradients Vanish?

For many randomly initialised deep variational circuits, researchers have shown

$$\boxed{\mathrm{Var}\left[\frac{\partial f}{\partial \theta}\right] = O(2^{-n})}$$

where n is the number of qubits.

Implications

- Gradient variance decreases exponentially
- Typical gradients become exponentially small
- Detecting them requires exponentially many measurement shots

Example

Qubits	Gradient Variance
10	$2^{-10} \approx 10^{-3}$
20	$2^{-20} \approx 10^{-6}$
30	$2^{-30} \approx 10^{-9}$

The parameter-shift rule gives the *exact* gradient, but if the true gradient is exponentially small, estimating it still requires exponentially many measurements.

Four Root Causes of Barren Plateaus

Root cause	Mechanism	Reference
Deep/random circuits	Circuit approaches a 2-design; information scrambled	McClean et al. [22]
Global measurements	Measuring all qubits $\equiv$ applying a deep circuit before a local measurement	Cerezo et al. [24]
Hardware noise	Noisy gates push state toward maximally mixed $I/2^n$	Wang et al. [25]
High entanglement entropy	Tracing out a large subsystem leaves nearly maximally mixed state	Marrero et al. [26]

All four share a common theme: *information scrambling*.

The Expressibility–Trainability Tension

Circuit	Expressibility	Trainability
Very shallow	Low	High
Shallow	Moderate	Good
Moderate	Good	Moderate
Deep	High	Poor
Very deep	Very high	**Fails**

The strongest model is not the deepest model.

The design principle: Find a *limited, structured* ansatz that restricts training to the *relevant subspace*.

Mitigation strategies:

- Local observables (not global)
- Problem-inspired shallow ansätze
- Identity initialisation ($\theta \approx 0$)
- Layer-by-layer greedy training

Table of Contents

The Naming Problem: "Quantum Neural Network"

Variational Quantum Circuits (VQCs) are widely called **Quantum Neural Networks (QNNs)** in the literature.

Why the name fits

- Trainable parameters
- Optimised by gradient descent
- Layered computation
- Learns from data by minimising a loss

Why the name is misleading

- No nonlinear activations between layers
- Quantum evolution is entirely linear
- No multi-layer perceptron structure
- Expressibility comes from a different mechanism

How is a VQC Different from a Classical Neural Network?

Classical Neural Network

$$h^{(l)} = \phi\left(W^{(l)} h^{(l-1)}\right)$$

- Linear layer
- Nonlinear activation after every layer
- Nonlinearity distributed throughout
- Without ϕ, many layers reduce to one linear map

Variational Quantum Circuit

$$f_\theta(x) = \langle \psi | M | \psi \rangle$$

- Every gate is unitary (linear)
- No activation between gates
- Measurement ($|a|^2$) provides the only nonlinearity
- Behaves more like a deep linear network

A Useful Interpretation: VQCs as Fourier Networks

$$f_\theta(x) = \sum_{\omega \in \Omega} c_\omega(\theta) e^{i\omega x}$$

This can be interpreted as a single-hidden-layer Fourier network.

Encoding determines

- Available frequencies Ω
- Hidden basis functions
- Feature vocabulary

Training determines

- Fourier coefficients $c_\omega(\theta)$
- Strength of each frequency
- Final learned function

Connection to Classical ML

This links VQCs to Fourier Neural Networks, Random Fourier Features (RFF), and kernel methods.

What Remains Genuinely Quantum?

Although VQCs admit useful classical interpretations, three ingredients remain uniquely quantum.

Quantum feature	Why it matters
Exponentially large Hilbert space	n qubits naturally represent a 2^n-dimensional state
Quantum interference	Probability amplitudes interfere constructively and destructively
Entanglement	Creates correlations beyond classical product-state models

Summary

In this module we answered three fundamental questions:

1. What functions can a VQC represent?
2. How can we compute exact gradients?
3. Why do highly expressive circuits become difficult to train?

The Big Picture

A Variational Quantum Circuit is a trainable quantum function approximator whose behaviour is determined by its **encoding**, **ansatz**, and **measurement**.

Practice Problem 1

A single-qubit VQC uses $L = 3$ repetitions of $R_x(x)$ encoding, each followed by a trainable Rot(θ) block, then a σ_z measurement.

(a) Using Theorem 5.1, state Ω and write $f_\theta(x)$ as an explicit sum of sinusoidal terms. How many independent sinusoidal components does the model have?

(b) The data contains a frequency component at $\omega = 5$. A student argues: "We can fit this by increasing the number of trainable parameters in Rot(θ) without adding more encoding layers."
Is the student correct? Justify your answer using the two-factor expressibility picture.

Practice Problem 1

(c) Write the parameter-shift rule for one parameter μ. Given

$$f\left(\frac{\pi}{6}+\frac{\pi}{2}\right) = -0.500, \qquad f\left(\frac{\pi}{6}-\frac{\pi}{2}\right) = 0.966,$$

compute

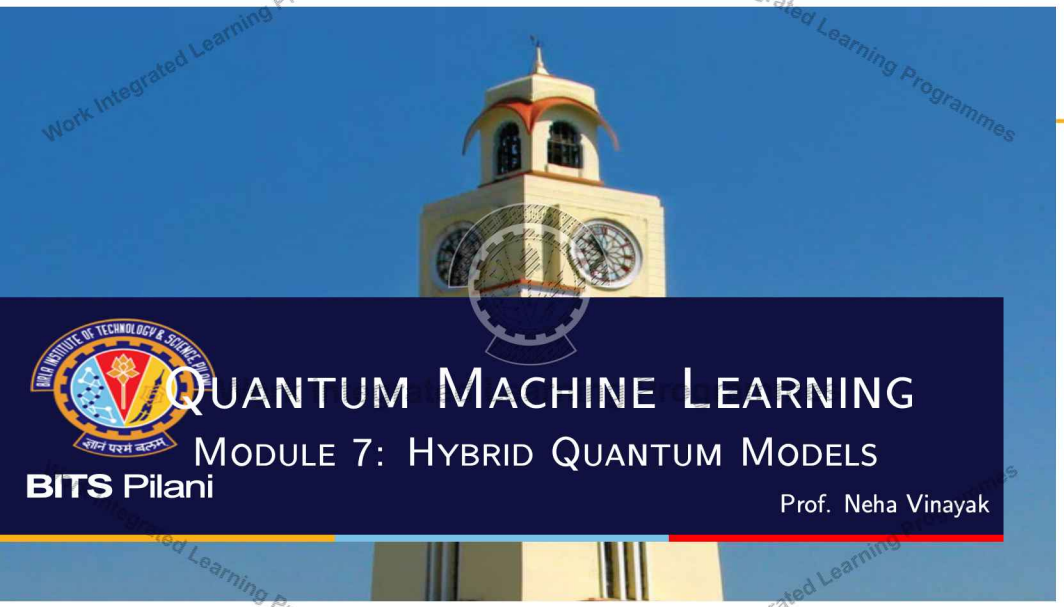
$$\left.\frac{\partial f}{\partial \mu}\right|_{\mu=\pi/6}.$$

(d) For a deep 25-qubit hardware-efficient ansatz,

$$\mathrm{Var}\left[\frac{\partial f}{\partial \theta}\right] \sim 2^{-25}.$$

Using the barren plateau result, explain why training becomes difficult. How does the required number of measurement shots scale with the number of qubits?

Thank you :)



Quantum Machine Learning

Module 7: Hybrid Quantum Models

BITS Pilani

Prof. Neha Vinayak

Table of Contents

Why Hybrid?

Classical networks excel at:

- High-dim inputs
- Large datasets, millions of parameters
- Mature tools: PyTorch, TensorFlow

Classical networks struggle with:

- Small labelled datasets - overfitting
- Exponential parameter growth

Quantum circuits offer:

- Hilbert space 2^n with only n qubits
- $O(n)$ parameters, not $O(2^n)$
- Entanglement

Hybrid strategy:

- Classical: compress high-dim data
- Quantum: process low-dim encoding
- Together: parameter-efficient models

Current devices: 50–100 usable qubits with high error rates. Quantum processing must receive clean, low-dimensional features from a classical front-end - not raw data.

Formal Definition of a Hybrid Model

Joint parameterisation of two parameter sets:

- θ_c - classical: dense layers, Q-Mapper weights
- θ_q - quantum: VQC rotation angles
- Single shared loss: $\min_{\theta_c,\theta_q} \mathcal{L}(\theta_c, \theta_q) = \frac{1}{N}\sum_{i=1}^{N} \ell(\hat{y}^{(i)}, y^{(i)})$

Two different gradient mechanisms:

Component	Params	Gradient method
Classical layers	θ_c	Backpropagation - one backward pass
Quantum circuit	θ_q	Parameter-shift - 2 circuits per param
CNN backbone	-	Frozen - zero gradient cost

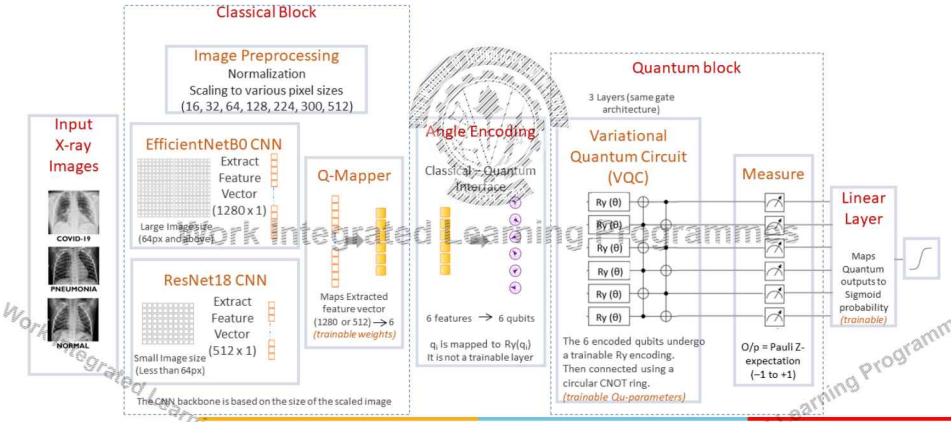
- Classical autograd cannot differentiate through a quantum device
- The VQC must be wrapped as a differentiable layer: the **quantum node**
- Both gradient streams merged by the classical optimiser (Adam, RMSProp)

Three Integration Patterns

Pattern	Data flow	Best for
Series (most common)	Classical encoder → VQC → head	High-dim data, NISQ; single quantum–classical boundary
Parallel (ensemble)	Multiple VQCs → classical aggregate	Uncertainty estimation; $k\times$ circuit cost
Interleaved (alternating)	Dense ↔ VQC ↔ Dense ⋯	Hybrid VQE, small-dim inputs; multiple noisy boundaries

Series Architecture: Case Study

QXRNet Architecture Vinayak & Ahmad, *Quantum Machine Intelligence* (2024)



Series Architecture - Stage by Stage

Stage	I/O dims	What it does
Classical Encoder	$\mathbb{R}^D \to \mathbb{R}^d$	CNN/PCA compresses raw input; handles high-dimensionality the quantum circuit cannot
Q-Mapper	$\mathbb{R}^d \to \mathbb{R}^n$	Linear projection $\phi = W_e z$; one angle per qubit; $nd+n$ backprop-trained params
Quantum Layer	$\mathbb{R}^n \to [-1,1]^n$	Encode with $R_Y(\phi_i) \to$ apply VQC $U(\theta) \to$ measure Pauli-Z expectation values $\langle Z_k \rangle$ to obtain q
Classical Head	$\mathbb{R}^n \to (0,1)$	$\hat{y} = \sigma(Wq + b)$; sigmoid maps logit to class probability

Design Decisions

Three design decisions fixed at this stage:

- **Qubit count** n**:** must satisfy $n \leq$ NISQ limit (tens of qubits)
- **Q-Mapper linear:** nonlinear Q-Mapper degrades performance (QXRNet ablation)
- **Encoding gate** R_Y**:** real-amplitude subspace; cleanest parameter-shift

QXRNet - A Series Hybrid in Practice

Task: Binary chest X-ray classification
Dataset: NIH ChestX-ray14; multiple resolutions

Why QXRNet is a good QML candidate:

- Medical imaging: small datasets, class-imbalanced
- Classical CNNs overfit on limited positive samples
- VQC with 18 parameters has far lower overfitting risk

Pipeline:

$$\text{Image} \xrightarrow{\text{CNN (frozen)}} z \in \mathbb{R}^{512} \xrightarrow{\text{Q-Map}} \phi \in \mathbb{R}^{6} \xrightarrow{\text{VQC}} q \in \mathbb{R}^{6} \xrightarrow{\text{Head}} \hat{y}$$

Quantum node boundary: from ϕ encoding through measurement of $\langle Z_k \rangle$.

Vinayak & Ahmad, *QXRNet* (2024)

QXRNet - Architecture

Architecture decisions:

Decision	Justification
ResNet18 / EffNetB0	Resolution-conditioned; avoids retraining one large backbone
6 qubits	Saturates at 6; more qubits raise coherence risk (ablation)
3 variational layers	Deeper circuits degrade - overfitting + decoherence (ablation)
Ring CNOT	$0 \to 1 \to 2 \to 3 \to 4 \to 5 \to 0$; no SWAP overhead on ring/heavy-hex
Backbone frozen	Gradient instability + 11M vs. 18 param scale mismatch

QXRNet - Parameter Count

Number of Trainable Parameters:

Layer	Params	Note
CNN backbone	0	Frozen pre-trained
Q-Mapper	$6d + 6$	$\approx 3{,}078$ (ResNet18, $d = 512$)
Variational QNN	18	3 layers $\times$ 6 qubits (R_Y per qubit)
Classical head	7	Linear $6 \to 1$ + bias
Total	$\mathbf{6d + 31 = 3103}$	vs. millions in classical CNN

QXRNet achieves AUC comparable to or better than classical baselines with 10× fewer trainable parameters - demonstrating parameter efficiency in the small-data regime.

Why Freeze the Classical Backbone?

Three reasons rooted in NISQ realities:

1. **Gradient signal degrades through the quantum layer**
   - Path: loss → head → VQC (param-shift) → Q-Mapper → backbone
   - Parameter-shift adds shot noise at each quantum evaluation
   - Signal unreliable before it reaches 11M backbone weights
2. **Parameter scale mismatch**
   - ResNet18: 11 million params; VQC: 18
   - Same learning rate: backbone updates swamp quantum updates
3. **Backbone is already well-trained**
   - ImageNet pre-training gives general-purpose features
   - Quantum layer only needs to *combine* features, not re-learn them

Table of Contents

The Integration Problem

What DL frameworks expect from any layer:

- **Forward:** deterministic function, input tensor → output tensor
- **Backward:** given upstream gradient, return gradients w.r.t. own params

Why a quantum circuit does not fit:

- Measurement is **stochastic** - output sampled from Born rule
- $\langle Z_k \rangle$ requires S shots to estimate - not one function call
- $\partial_{\theta_k} \langle Z_k \rangle$ needs 2 circuit runs at $\theta_k \pm \pi/2$ - not a backward pass

Required: wrap the VQC so that:

- Forward pass returns $q \in \mathbb{R}^n$ (expectation values)
- Backward computes Jacobian $J \in \mathbb{R}^{n \times K}$ via parameter-shift
- J is returned to the optimiser as if from a standard layer

The Quantum Node Abstraction

The qnode contract:

Direction	What happens
Forward (in)	Classical $\phi \in \mathbb{R}^n$ (angles from Q-Mapper)
Inside	(i) Encode: $R_Y(\phi_i)\|0\rangle$ (ii) Evolve: $U(\theta_q)$ (iii) Measure: $\langle Z_k \rangle$
Forward (out)	Classical $q \in \mathbb{R}^n$ (expectation values)
Backward	Jacobian $J_{kj} = \frac{1}{2}[q_k(\theta_j + \pi/2) - q_k(\theta_j - \pi/2)]$ returned to autograd

From the framework's perspective:

- Just a function $q = f_{\theta_q}(\phi)$: $\mathbb{R}^n \to \mathbb{R}^n$
- Parameters θ_q registered with optimiser like any layer
- Only difference: Jacobian needs $2K$ circuit evaluations, not 1 pass

What the qnode hides: shot-based estimation, parameter-shift procedure, hardware transpilation, noise mitigation.

Forward Pass - Full Pipeline Trace

Four stages (QXRNet):

Stage	I/O	Notes
1. CNN + GAP	$x \to z \in \mathbb{R}^{512}$	Frozen; deterministic; no gradient computed
2. Q-Mapper	$z \to \phi \in \mathbb{R}^6$	Linear $\phi = W_e z + b_e$; trained by backprop
3. Quantum node	$\phi \to q \in [-1, 1]^6$	Encode $R_Y(\phi_i)$; variational layers; $\langle Z_k \rangle$ estimated from S shots - **only stochastic stage**
4. Head	$q \to \hat{y} \in (0, 1)$	$\hat{y} = \sigma(W_h q + b_h)$; binary: $\hat{y} > 0.5 \Rightarrow$ positive

Backward Pass - Three Gradient Zones

Loss: $\mathcal{L} = -[w_1 y \log \hat{y} + w_0(1-y)\log(1-\hat{y})]$, $\delta_a = \hat{y} - y$

Zone	Params	Gradient computation
Zone 1: Head	W_h, b_h	Backprop: $\nabla_{W_h}\mathcal{L} = \delta_a q^\top$
Zone 2: Q-Mapper	W_e, b_e	Upstream $\delta_q = W_h^\top \delta_a$ flows through quantum Jacobian J; backprop into W_e
Zone 3: VQC	θ_q	$J_{kj} = \frac{1}{2}[q_k^+ - q_k^-]$; $\nabla_{\theta_j}\mathcal{L} = \delta_q^\top J_{\cdot j}$
CNN	-	Frozen - zero gradient cost

The gradient handoff

$\delta_q = W_h^\top \delta_a$ is classical (free). J is computed by parameter-shift on the quantum device ($2K$ runs). $\nabla_{\theta_q}\mathcal{L} = J^\top \delta_q$ is a classical product - neither side needs to know the other's internals.

The Gradient Gluing Point

Chain rule at the quantum–classical boundary:

$$\frac{\partial \mathcal{L}}{\partial \theta_j} = \sum_{k=1}^{n} \underbrace{\frac{\partial \mathcal{L}}{\partial q_k}}_{\delta_q^{(k)}\ \text{(classical)}} \cdot \underbrace{\frac{\partial q_k}{\partial \theta_j}}_{J_{kj}\ \text{(quantum)}} = \delta_q^\top J_{\cdot j}$$

Classical side - $\delta_q = W_h^\top \delta_a \in \mathbb{R}^n$: one vector-matrix product via autograd (free).

Quantum side - parameter-shift:

$$J_{kj} = \frac{1}{2}\left[q_k(\theta_j + \tfrac{\pi}{2}) - q_k(\theta_j - \tfrac{\pi}{2})\right]$$

QXRNet: $K = 18$ VQC params $\Rightarrow$ 36 circuit runs to fill $J \in \mathbb{R}^{6\times 18}$.

Glue step (classical): $\nabla_{\theta_q}\mathcal{L} = J^\top \delta_q \in \mathbb{R}^K$ PennyLane returns J to PyTorch autograd $\rightarrow$ multiplied $\rightarrow$ passed to Adam.

Shot Budget - The Hidden Cost

Operation	Circuit runs	Shots (S=1000)
Forward pass	1	1,000
Jacobian (2 × 18 params)	36	36,000
Q-Mapper grad (classical)	0	0
CNN (frozen)	0	0
Total per step	**37**	**37,000**

Simulation vs. real hardware:

- **Statevector simulation** (QXRNet): exact $\langle Z_k \rangle$, $S = \infty$, microseconds/run - used for all training
- **Real hardware:** $S = 1000 \approx 1$ s per run; 37 runs/step × 500 steps ≈ 5 h QPU per epoch
- **Shot noise:** $\sigma \approx 1/\sqrt{S}$; at $S = 100$, $\sigma \approx 0.1$ - can swamp small gradients

Example - Forward Pass Through a 2-Qubit Node

Setup: $\phi = [\pi/4,\ \pi/3]$; $\theta_q = [0, 0]$; CNOT (ctrl: q_0, tgt: q_1).

Step 1 - State after encoding:

$$|\psi_0\rangle = (\cos\tfrac{\pi}{8}|0\rangle + \sin\tfrac{\pi}{8}|1\rangle) \otimes (\cos\tfrac{\pi}{6}|0\rangle + \sin\tfrac{\pi}{6}|1\rangle)$$

Values: $\cos(\pi/8) \approx 0.924$, $\sin(\pi/8) \approx 0.383$, $\cos(\pi/6) \approx 0.866$, $\sin(\pi/6) = 0.500$

Step 2 - Apply CNOT:

$$|\psi_1\rangle \approx 0.800|00\rangle + 0.462|01\rangle + 0.332|11\rangle + 0.192|10\rangle$$

Step 3 - Compute $\langle Z_0 \rangle$:

$$\langle Z_0 \rangle = (0.800^2 + 0.462^2) - (0.192^2 + 0.332^2) = 0.854 - 0.147 \approx \mathbf{0.707}$$

Entanglement encodes cross-feature information independent neurons cannot.

Backward Pass - Quantum Jacobian

Continuing the 2-qubit example. At baseline $\theta_q = [0, 0]$: $q = [\langle Z_0 \rangle, \langle Z_1 \rangle] = [0.707,\ 0.354]$.

Jacobian $J \in \mathbb{R}^{2\times 2}$: $J_{kj} = \frac{1}{2}[q_k(\theta_j + \frac{\pi}{2}) - q_k(\theta_j - \frac{\pi}{2})]$

Column 1 - shift θ_0 (fix $\theta_1 = 0$):

$$q(\theta_0 + \tfrac{\pi}{2}) = [-0.612,\ 0.354], \qquad q(\theta_0 - \tfrac{\pi}{2}) = [+0.612,\ 0.354]$$

$$J_{\cdot 0} = \tfrac{1}{2}\big([-0.612, 0.354] - [0.612, 0.354]\big) = [-0.612,\ 0.000]$$

Column 2 - shift θ_1 (fix $\theta_0 = 0$):

$$q(\theta_1 + \tfrac{\pi}{2}) = [0.707,\ -0.512], \qquad q(\theta_1 - \tfrac{\pi}{2}) = [0.707,\ +0.512] \text{ (approx)}$$

$$J_{\cdot 1} = \tfrac{1}{2}\big([0.707, -0.512] - [0.707, 0.512]\big) = [0.000,\ -0.866]$$

$$J = \begin{pmatrix} -0.612 & 0.000 \\ 0.000 & -0.866 \end{pmatrix}$$

Backward Pass - Gradient of Loss

Setup: Head weight $W_h = [1, 1]$, bias $= 0$, target $y = 1$, loss $=$ BCE.

Step 1 - Classical head forward:

$$a = W_h\, q + 0 = 0.707 + 0.354 = 1.061, \qquad \hat{y} = \sigma(1.061) \approx 0.743$$

$$\mathcal{L} = -\log(0.743) \approx 0.297$$

Step 2 - Error at head output (classical backprop):

$$\delta_a = \hat{y} - y = 0.743 - 1 = -0.257$$

Step 3 - Error propagated to quantum outputs:

$$\delta_q = W_h^\top\, \delta_a = \begin{pmatrix} 1 \\ 1 \end{pmatrix} (-0.257) = \begin{pmatrix} -0.257 \\ -0.257 \end{pmatrix}$$

Backward pass - Gradient of Loss

Step 4 - Gradient of loss w.r.t. VQC parameters (glue step):

$$\nabla_{\theta_q} \mathcal{L} = J^\top\, \delta_q = \begin{pmatrix} -0.612 & 0.000 \\ 0.000 & -0.866 \end{pmatrix}^\top \begin{pmatrix} -0.257 \\ -0.257 \end{pmatrix} = \begin{pmatrix} +0.157 \\ +0.222 \end{pmatrix}$$

Step 5 - Adam update ($\eta_q = 10^{-3}$, first step $\Rightarrow$ Adam $\approx$ SGD):

$$\theta_0 \leftarrow 0 - 10^{-3}(+0.157) = -0.000157, \qquad \theta_1 \leftarrow 0 - 10^{-3}(+0.222) = -0.000222$$

Interpretation: Both gradients are positive, so both θ_0 and θ_1 decrease. $|\partial\mathcal{L}/\partial\theta_1| > |\partial\mathcal{L}/\partial\theta_0|$ because $|\partial\langle Z_1\rangle/\partial\theta_1| = 0.866 > 0.612 = |\partial\langle Z_0\rangle/\partial\theta_0|$ - qubit 1 has larger leverage on the loss.

Table of Contents

The Two-Parameter-Family Problem

Property	Classical θ_c	Quantum θ_q
Gradient cost	1 backward pass	$2K$ circuit runs
Gradient noise	None (simulation)	Shot noise $\sigma \sim 1/\sqrt{S}$
Parameter count	3,078 (Q-Mapper)	18 (VQC)
Landscape	Smooth, well-understood	Periodic; barren plateau risk
Gradient range	Unconstrained	Bounded $[-0.5, +0.5]$ (PS)

The mismatch problem and solution:

- Same learning rate for both sides is risky - magnitudes are incomparable
- **Solution:** separate `param_groups` in the optimiser
- QXRNet: $\eta_c = 10^{-4}$ (classical), $\eta_q = 10^{-3}$ (quantum)
- Quantum LR larger because parameter-shift bounds gradient to $[-0.5, +0.5]$

Optimiser Choices for the Quantum Side

- **COBYLA / Nelder-Mead (gradient-free):**
  - Evaluates $\mathcal{L}(\theta_q)$ at trial points; no gradient needed
  - Scales as $O(K^2)$ evaluations for K parameters
  - Best for: $K < 20$, real noisy hardware
- **SPSA (2-circuit gradient approximation):**
  - Random perturbation $\Delta \in \{-1, +1\}^K$; approximates gradient with 2 runs: $\hat{\nabla}_{\theta_j}\mathcal{L} \approx [\mathcal{L}(\theta + c\Delta) - \mathcal{L}(\theta - c\Delta)]/(2c\Delta_j)$
  - Fixed cost of 2 circuit runs regardless of K - shot-frugal
- **Parameter-shift + Adam (gold standard in simulation):**
  - Exact quantum gradients; Adam handles periodic landscape

Recap: Adam Optimiser

Standard SGD Issues: Same learning rate for all parameters; No memory

Adam fixes both with two running averages:

Quantity	Formula	What it tracks
m_t (1st moment)	$\beta_1 m_{t-1} + (1-\beta_1) g_t$	Smoothed gradient direction
	typical: $\beta_1 = 0.9$	(EMA of past gradients)
v_t (2nd moment)	$\beta_2 v_{t-1} + (1-\beta_2) g_t^2$	Smoothed gradient magnitude
	typical: $\beta_2 = 0.999$	(EMA of past squared gradients)
$\hat{m}_t$, $\hat{v}_t$	divide by $(1-\beta^t)$	Bias correction for cold start

Final update: $\theta_t = \theta_{t-1} - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$

Adam Optimiser - Numerical Example

Setup: QXRNet quantum layer. One parameter θ_1 (first qubit, first layer). Adam hyperparameters: $\eta = 10^{-3}$, $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$.

Adam update equations:

$$m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t, \qquad v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2$$

$$\hat{m}_t = \frac{m_t}{1-\beta_1^t}, \qquad \hat{v}_t = \frac{v_t}{1-\beta_2^t}, \qquad \theta_t = \theta_{t-1} - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$$

Numerical trace (initialise $m_0 = v_0 = 0$, $\theta_0 = 1.200$):

t	g_t	m_t	v_t	$\hat{m}_t$	$\hat{v}_t$	θ_t
1	−0.482	−0.0482	0.000232	−0.4820	0.2323	1.201
2	−0.455	−0.0889	0.000439	−0.4678	0.2197	1.202
3	−0.371	−0.1171	0.000576	−0.4321	0.1923	1.203

Adam Worked Example - Analysis of Results

Revisit the three-step trace:

t	g_t	m_t	v_t	$\hat{m}_t$	$\hat{v}_t$	$\Delta\theta_t$
1	-0.482	-0.0482	0.000232	-0.482	0.232	$+0.001000$
2	-0.455	-0.0889	0.000439	-0.468	0.220	$+0.000998$
3	-0.371	-0.1171	0.000576	-0.432	0.192	$+0.000986$

Why $\beta_2 = 0.999$ for VQCs?

Quantum gradients from the shift rule are unbiased but noisy. $\beta_2 = 0.999$ gives very slow adaptation of v_t, averaging over ≈ 1000 steps before v_t stabilises. This prevents a few noisy gradient spikes from permanently shrinking the step size.

Why Adam Suits VQC Training

The periodic landscape problem: VQC loss $\mathcal{L}(\theta)$ is periodic - $\mathcal{L}(\theta+2\pi) = \mathcal{L}(\theta)$. Gradients can be tiny one step and moderate the next.

How Adam helps:

- m_t smooths noisy shot-based gradient estimates
- v_t adapts the step size per parameter
- Bias correction prevents artificially small steps at $t = 1$ when $m_0 = v_0 = 0$

Numerical comparison at step 1 ($g_1 = -0.482$, $\theta_0 = 1.200$):

Optimiser	Effective step size	θ_1
SGD ($\eta = 10^{-3}$)	$10^{-3} \times 0.482 = 0.000482$	1.200482
Adam ($\eta = 10^{-3}$)	$10^{-3} \times 0.482/\sqrt{0.232} = 0.001$	1.201

Adam takes a $2\times$ **larger effective step** when $\hat{v}_t$ is small

When Does Hybrid Training Win?

Conditions where hybrid models have an advantage:

- Small labelled datasets
- Low-dimensional feature space after encoding
- Class imbalance / rare events
- Non-linear decision boundaries in compressed space

Where hybrid does *not* help:

- Large datasets ($N \gg 10{,}000$)
- Features not compressible to qubit count
- Deep circuits required for expressibility

Current hybrid models demonstrate *parameter efficiency* in the small-data regime, not provable computational speedup over classical models.

Table of Contents

The NISQ Reality Check

Parameter	What it means for your hybrid model
Gate fidelity	Two-qubit CNOT: 99–99.5% on best devices; each CNOT has 0.5–1% chance of error. Single-qubit: 99.9%
T1 (energy relaxation)	$\|1\rangle \to \|0\rangle$ decay; typical 50–200 μs. Circuit must complete well before T1
T2 (dephasing)	Superposition phase randomises; typical 30–150 μs; always T2$\leq$2T1. Binding coherence constraint
Qubit connectivity	Not all qubit pairs connected; IBM heavy-hex: max 3 neighbours. Non-adjacent CNOT needs SWAP routing (3 extra CNOTs)

Qubit count, circuit depth, and entanglement pattern must all be chosen with these hardware parameters in mind - not just expressibility.

Circuit Depth as the Critical Constraint

Error accumulation: $P(\text{error}) = 1 - (1-p)^G$
At $p = 0.01$, $G = 50$: $P \approx 39.5\%$ - nearly 40% of circuit runs corrupted.

Two constraints (T2=100 μs, t_g=200 ns, $p = 0.01$):

Constraint	Gate limit
T2 coherence: $d_{\max} = T_2/t_g$	500
Gate error ($P < 50\%$): $G < 69$	69

Keep total two-qubit gate count below ~50–70 on NISQ hardware.

Hardware-Efficient Ansatz (HEA)

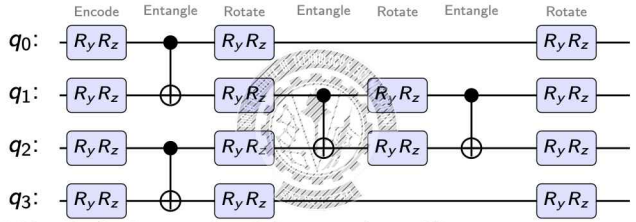
Design principles

- **Native gates only:** no translation overhead (IBM: CNOT/ECR; IonQ: MS gate)
- **Respect connectivity:** two-qubit gates only between adjacent qubits
- **Brick-wall pattern:** alternate even-odd / odd-even pairs per layer

Layer 1: CNOTs on pairs (0,1) and (2,3). Layer 2: CNOT on pair (1,2).

Alternating CNOT pattern spreads entanglement using only nearest-neighbour interactions - directly implementable on linear or ring qubit chains.

Structure of One HEA Layer (4 Qubits)



CNOT pattern: Step 1 - $q_0 \to q_1$ and $q_2 \to q_3$ (two parallel CNOTs, even layer).
Step 2 - $q_1 \to q_2$ (single CNOT, odd layer).
Step 3 - $q_1 \to q_2$ again (as read from image).
Trainable parameters: 20 (for rotation gates) **CNOT gates are fixed** - no parameters.

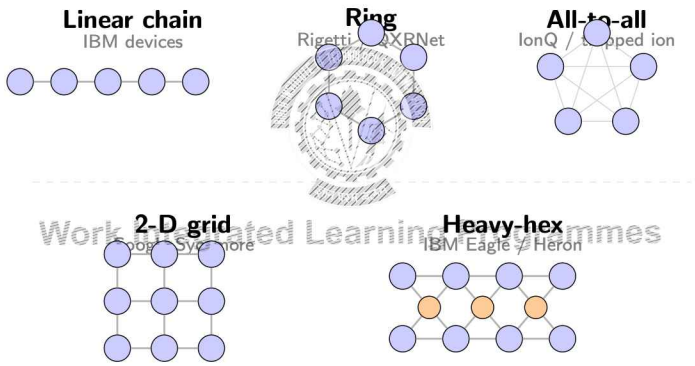
Qubit Connectivity Constraints

SWAP overhead:

- Adjacent CNOT: **1 gate**, 1 error
- $\text{SWAP}(q_i, q_j) = \text{CNOT}_{ij} \cdot \text{CNOT}_{ji} \cdot \text{CNOT}_{ij}$ = **3 CNOTs**
- Non-adjacent CNOT (2 hops): $3 + 1 = 4$ CNOTs total

Topology	Example	Implication
Linear chain	IBM devices	Nearest-neighbour CNOTs only
Heavy-hex	IBM Eagle	Max 3 neighbours; brick-wall fits naturally
All-to-all	IonQ	Any CNOT native; full entanglement fine
Ring	Some Rigetti	Ring CNOT maps directly

Qubit Connectivity Topologies



Zero-Noise Extrapolation (ZNE)

Idea: measure at amplified noise levels and extrapolate to zero noise.

1. Run circuit at noise levels $\lambda = 1, 2, 3$ using **gate folding**: replace G with $G \cdot G^\dagger \cdot G$ (ideal circuit unchanged; physical noise multiplied)
2. Fit linear model: $\langle Z \rangle_\lambda = a + b\lambda$
3. Extrapolate: $\langle \hat{Z} \rangle_0 = a$ (intercept)

Cost and limits:

- r noise levels $\Rightarrow$ $r\times$ circuit evaluations per step
- Works best: shallow circuits, near-linear noise, signal $>$ shot noise floor
- ZNE vs PEC: ZNE cheaper (black-box, no noise model); PEC more accurate but exponentially expensive ($\propto \gamma^{2G}$)

Worked Example - ZNE Linear Extrapolation

Measured $\langle Z_0 \rangle$ at three noise levels: $\lambda = 1$: 0.72; $\lambda = 2$: 0.58; $\lambda = 3$: 0.44; Linear model: $\langle Z_0 \rangle(\lambda) = a + b\lambda$

Step 1 - Fit using $\lambda = 1$ and $\lambda = 3$:

$$a + b = 0.72 \quad \text{and} \quad a + 3b = 0.44 \;\Rightarrow\; 2b = -0.28 \;\Rightarrow\; b = -0.14,\; a = 0.86$$

Step 2 - Verify at $\lambda = 2$: $0.86 + (-0.14) \times 2 = 0.58$ ✓

Step 3 - Extrapolate to $\lambda = 0$:

$$\langle \hat{Z}_0 \rangle_0 = a = \mathbf{0.86}$$

Interpretation:

- Raw hardware: 0.72; ZNE estimate: 0.86
- Without ZNE: 16% underestimate - biases the decision boundary
- Cost: 2 extra noise-level circuit runs per expectation value

Table of Contents

Three Types of Quantum Noise

Type	Physical origin	Correctable by optimiser?
Coherent error	Systematic mis-rotation: $R_Y(\theta+\delta)$ instead of $R_Y(\theta)$	**Yes** (OPR): optimiser converges to $\theta^* = \theta_{\text{true}} - \delta$
T1 amplitude damping	$\|1\rangle \to \|0\rangle$ decay; rate $\Gamma_1 = 1/T_1$; irreversible	**No**: random per shot
T2 phase damping	Phase randomises; rate $\Gamma_2 = 1/T_2$; T2≤2T1	**No**: random per shot
SPAM (State Preparation and Measurement) error	Readout: $\|0\rangle$ read as $\|1\rangle$; $\epsilon \approx 1$–5% per qubit	Partially (calibration matrix)

Noise Impact on the Cost Landscape

How it happens:

- Hardware noise and decoherence cause the quantum state to gradually lose information
- Expectation values $\langle Z_k \rangle$ shrink toward zero as circuit depth increases
- The cost landscape becomes flatter — gradients approach zero

This is called a **noise-induced barren plateau**.

	Barren plateau	Noise-induced BP
Cause	Deep random circuits	Decoherence / gate errors
Scales with	Qubit count n	Circuit depth d
In simulation?	Yes	No
Fix	Shallow, structured ansatz	Shallow circuits + ZNE

Mitigation: Keep circuits shallow. Apply **Zero-Noise Extrapolation (ZNE)**

Worked Example - Gate Error Accumulation

Setup: 4-qubit VQC, d layers; 12 single-qubit gates + 3 CNOTs per layer. $p_1 = 0.001$, $p_2 = 0.01$.

$$P(\text{error}) = 1 - (1-p_1)^{12d}(1-p_2)^{3d}$$

d	G_{1q}	G_{2q}	$P(\text{error})$	Verdict
3	36	9	$\approx$ **11.9%**	Manageable
6	72	18	$\approx$ **22.3%**	High
10	120	30	$\approx$ **34.4%**	Very high

Interpretation:

- $d = 3$: 12% of runs corrupted
- Doubling depth nearly doubles error probability
- ZNE with $r = 3$ recovers $\sim 12\%$ suppression at $3\times$ circuit cost

Table of Contents

Problem Setup and Dataset

ULB Credit Card Fraud (Kaggle):

- 284,807 transactions; 492 fraud (0.17%)
- Features: V1–V28 (PCA), Amount, Time - 30 total
- Naive predictor "always normal": 99.83% accuracy, **zero fraud detected**

QML pipeline design (5 steps):

Step 1	Data encoding: feature selection, normalisation, encoding gate
Step 2	Circuit design: ansatz, qubit count, entanglement
Step 3	Training option: variational vs. quantum kernel
Step 4	Training workflow: loss, CV, optimiser, stopping
Step 5	NISQ limitations and mitigation strategies

Steps 1 & 2: Encoding and Circuit Design

Step 1 - Encoding:

- Select V1–V6 (highest PCA variance; V7–V28 low discriminative value)
- Rescale: to avoid 2π-periodicity wrapping
- Gate: $R_Y(\phi_i)|0\rangle$ on 6 qubits; chosen over R_X, R_Z by ablation

Step 2 - Circuit design:

- 6-qubit HEA, 3 variational layers of $R_Y(\theta)$ per qubit
- Ring CNOT entanglement: $0 \to 1 \to 2 \to 3 \to 4 \to 5 \to 0$
- Pauli-Z measurement: $q = (\langle Z_1 \rangle, \ldots, \langle Z_6 \rangle)$

Component	Params
VQC (3×6 rotation gates)	18
Q-Mapper ($6 \times 6 + 6$)	42
Classical head ($6 \times 1 + 1$)	7
Total	**67**

Steps 3, 4 & 5: Training and NISQ Limits

Step 3 - Two options:

- **Variational** (this session): train θ_q with param-shift + wBCE
- **Kernel SVM** (Module 8): $\kappa(x, x') = |\langle \psi(x) | \psi(x') \rangle|^2$; at $N = 284{,}807$: $\sim 10^{10}$ circuit evals - **infeasible**

Step 4 - Workflow: SMOTE training fold; $w_1 = 578$; 5-fold CV; early stopping.

Step 5 - NISQ limitations:

- **Shot budget:** 37,000 shots/step; ≈324M total; ≈90h QPU - *simulate*
- **Scalability:** adding features = more qubits + deeper circuits; 12 features doubles $P(\text{error})$
- **Noise:** $P(\text{error}) \approx 12\%$ at $d = 3$; ZNE with $r = 3$ recovers suppression
- **No speedup:** benefit is parameter efficiency, not computation

Practice Problem: Gold Price Prediction

Scenario: Predict gold price direction (up/down) from past 5 days $\times$ 4 features $= 20$ inputs. Dataset: 500 samples; classical MLP and LSTM overfit severely.

(a) The team proposes a 20-qubit VQC. Explain why this violates two specific NISQ constraints. Propose a dimensionality reduction step and recommend a qubit count with justification.

(b) Design a complete VQC for your qubit count from (a). Specify: (i) encoding gate and angle formula; (ii) ansatz (layers, gate types, entanglement) with justification; (iii) measurement observable and how binary $\hat{y}$ is obtained.

(c) During training the loss oscillates and does not converge. Identify two distinct causes from the hybrid training perspective (not barren plateaus). Propose one concrete mitigation per cause.

(d) Variational training or quantum kernel SVM? Justify using $N = 500$ and the likely nature of the decision boundary.

Practice Problem: Stock Movement Prediction

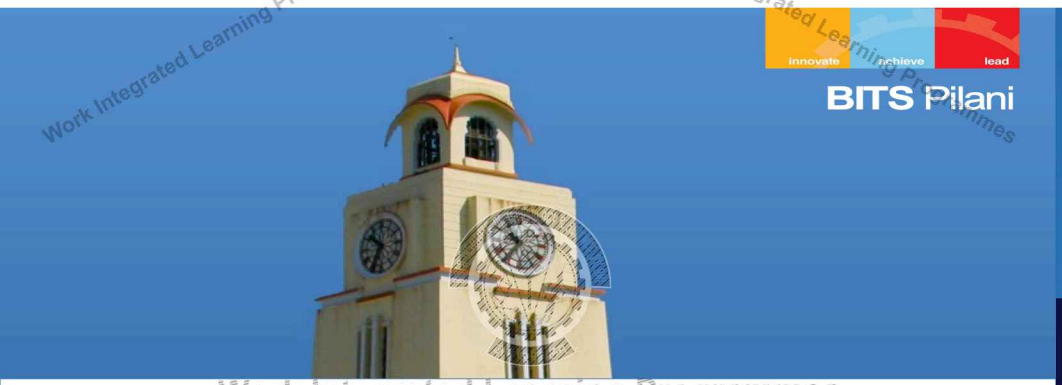
Scenario: Predict next-day price direction from 8 technical indicators (RSI, MACD, Bollinger, volume ratio, EMA-5, EMA-20, momentum, volatility). Dataset: 800 samples; highly non-linear relationships.

(a) Propose a complete series hybrid architecture. Draw a block diagram with four labelled stages and input/output dimensions. Justify qubit count.

(b) Write forward propagation equations: $z = W_{\text{pre}}x + b_{\text{pre}}$; $\phi = W_e z + b_e$; $q_k = \langle Z_k \rangle_{\theta_q}(\phi)$; $\hat{y} = \sigma(W_h q + b_h)$. State the dimensions of every weight matrix and intermediate vector.

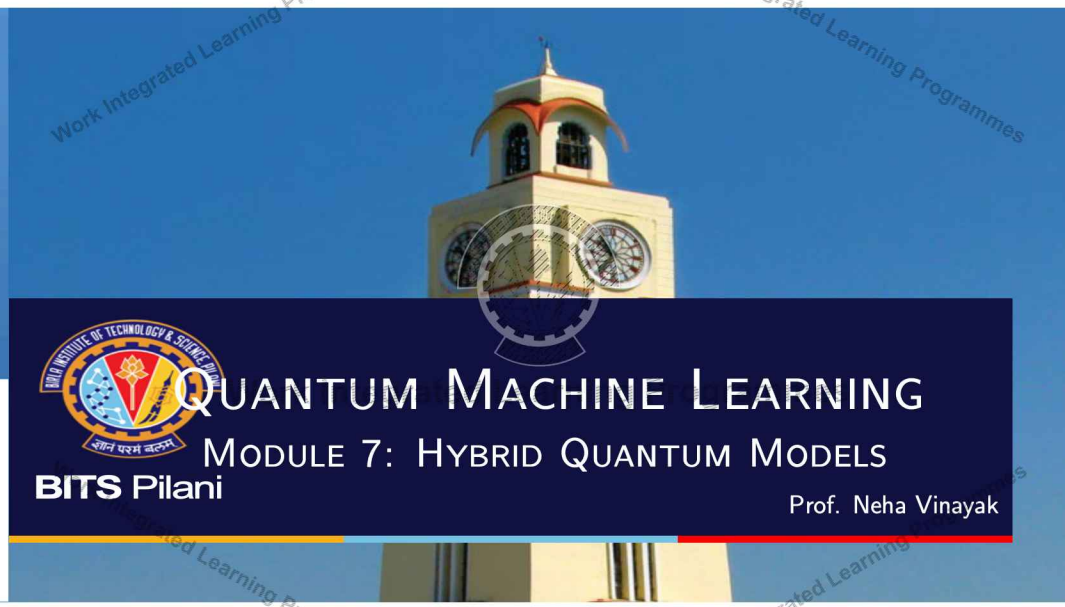
(c) Identify two hardware-aware design choices. For each: (i) NISQ constraint addressed; (ii) specific design decision; (iii) what fails if not made.

(d) Model achieves AUC $= 0.71$ vs. AUC $= 0.69$ for classical MLP with $10\times$ more parameters. Is this meaningful? What two pieces of evidence are needed to claim quantum advantage?



BITS Pilani

Thank you :)



Quantum Machine Learning

Module 7: Hybrid Quantum Models

BITS Pilani

Prof. Neha Vinayak

Table of Contents

When Does HEA Become a Poor Design?

Suppose your input is a 1024-qubit quantum state from a quantum sensor
You want to classify it (e.g. which phase of matter it belongs to).

HEA approach:

- 1024 qubits, 3 variational layers
- Every layer applies gates to all 1024 qubits - no locality, no hierarchy
- Global observable on all qubits, Barren Plateau risk

The HEA has no notion of: Local correlations, Hierarchical abstraction, Weight sharing

These are exactly the properties that make classical CNNs powerful.

Can we build a quantum circuit that scales like a CNN?

Cong, Choi & Lukin (2019) answer this

Why It Is Called a *Quantum* CNN

Concept	Classical CNN	QCNN
Local operation	3×3 filter sees 9 pixels at a time	2-qubit unitary $U(\theta)$ sees 2 neighbouring qubits
Weight sharing	Same filter weights applied to every patch	Same $U(\theta)$ applied to every qubit pair
Pooling	Max or average pool halves spatial size	Mid-circuit measurement halves active qubit count
Hierarchical rep.	Low layers: edges; high layers: objects	Low rounds: local correlations; high rounds: global phase
Output	Scalar class score via fully-connected head	Measurement on 1 final qubit; class label

QCNN Architecture: Three Layer Types

1. Convolutional layer (C): A parameterised 2-qubit unitary $U(\theta)$ applied to all pairs of adjacent qubits.

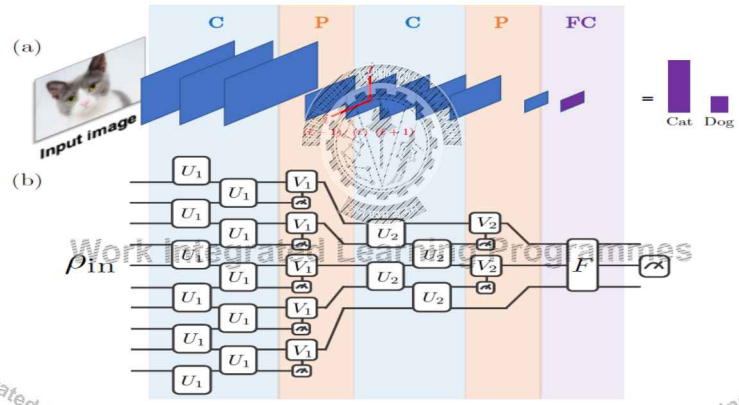
Conv: $U(\theta)$ on (q_0, q_1), $U(\theta)$ on (q_2, q_3), ... (same θ throughout)

2. Pooling layer (P): A subset of qubits is measured mid-circuit. Their measurement outcomes (classical bits m_k) control correction unitaries $V_j(m_k)$ on the surviving neighbours. Active qubit count halves.

3. Fully connected layer (FC): After $\log_2 N$ rounds of C+P, the system is reduced to $O(\log N)$ qubits. Then a general unitary $F(\theta_{\text{FC}})$ is applied.

Layer	Qubit count	Parameter property
Conv	n (unchanged)	Same θ for all pairs
Pool	$n \to n/2$	Correction V_j per outcome
FC	$O(\log N)$	General unitary

QCNN Structure



Pooling via Mid-Circuit Measurement

QCNN Pooling step by step:

1. **Measure** qubit q_k in the computational basis. Result: classical bit $m_k \in \{0, 1\}$. q_k is now collapsed - removed from the active circuit.
2. **Feed forward** m_k classically to the next gate.
3. **Apply correction** unitary $V_j(m_k)$ to the surviving neighbouring qubit q_j. This is a classically-controlled quantum operation - it injects information from the measured qubit into the survivor.

What is achieved?	Why it matters
Dimension reduction	q_k is gone; active qubit count halves - the same purpose as max-pool
Information transfer	m_k carries partial information about q_k's state; $V_j(m_k)$ injects it into the surviving qubit - unlike max-pool which simply discards

Parameter Efficiency: Cong's QCNN

Cong's QCNN parameter count is logarithmic:
Each pooling layer halves the active qubits, so only $\log_2 N$ convolution-pooling stages are required. One shared unitary $U(\theta)$ is learned at each stage.

N (qubits)	QCNN rounds ($\log_2 N$)	HEA gate groups ($N/2 \times L$, $L = 3$)	Ratio
8	3	12	4×
16	4	24	6×
64	6	96	16×
1024	10	1536	154×

QCNN can handle large quantum systems where HEA would face severe barren plateau risk.

No Barren Plateaus: Why QCNN Trains

Pesah et al. (2021): For QCNNs with local cost functions (measuring 1–2 final qubits), gradient variance decays *polynomially* with N, not exponentially like HEA:

$$\text{Var}\left[\frac{\partial \mathcal{L}}{\partial \theta}\right] = \Omega\left(\frac{1}{\text{poly}(N)}\right)$$

Why? Three structural reasons:

1. Each convolutional gate $U(\theta)$ acts on 2 neighbouring qubits. Its gradient depends only on the *reduced state of those 2 qubits*, not on the full 2^N-dimensional Hilbert space.
2. The cost function sees the final qubit's state.
3. Mid-circuit measurement breaks global mixing.

A 1024-qubit QCNN is trainable. A 1024-qubit HEA with global cost is not.